

LAPORAN PRAKTIKUM BIG DATA

PRAKTIKUM 4

NoSQL dan Pemodelan Data

Satu Data Banyak Cara Menyimpan



Disusun oleh:

Habibi Putra Rizqullah

NIM 2411531001

Kelas A

Dosen Pengampu:

Afdhal Dinilhak, M.Kom, S.Kom

FAKULTAS TEKNOLOGI INFORMASI

PROGRAM STUDI INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
BAB II DASAR TEORI.....	2
2.1 NoSQL dan Pemilihan Model Data	2
2.2 Tabel Relasional dan Key Value.....	2
2.3 Dokumen Bersarang dan Fleksibilitas Skema	2
2.4 Parquet dan Query Analitik DuckDB.....	2
2.5 Graph Berarah dan Keterjangkauan.....	2
2.6 Indeks Spasial STRtree.....	2
2.7 Delta Lake dan Riwayat Transaksi	3
2.8 Posisi Praktikum dalam Pipeline Big Data	3
BAB III METODOLOGI PRAKTIKUM.....	4
3.1 Waktu dan Lingkungan Praktikum.....	4
3.2 Perangkat dan Perangkat Lunak.....	4
3.3 Dataset.....	4
3.4 Akses Notebook dan Repository	5
3.5 Tahapan Praktikum.....	6
BAB IV HASIL DAN PEMBAHASAN.....	7
4.1 Menyiapkan Lingkungan dan Data Contoh K-1	7
4.2 Tabel Biasa dan Key Value K-2	11
4.3 Document dan Fleksibilitas Data K-3	13
4.4 DuckDB pada Parquet Berpartisi K-4.....	15
4.5 Graph dan Keterhubungan Antarzona K-5	18
4.6 Indeks Spasial K-6.....	21
4.7 Delta Lake Versi Time Travel dan Skema K-7	22
4.8 Small File Problem dan Compaction K-8	25
4.9 Rekap Benchmark K-9	26
4.10 Analisis Hasil.....	28
4.11 Studi Kasus Rancangan Penyimpanan Aplikasi Taksi	30
4.11.1 Aliran Data dan Kebutuhan Lokasi.....	31
4.11.2 Data Ganda dan Sinkronisasi.....	32
4.12 Latihan Praktikum	33
4.12.1 Pencarian 2000 ID	33
4.12.2 Query Document dan SQL.....	33
4.12.3 SUM Satu dan Beberapa Kolom	35
4.12.4 Out Degree Graph dan SQL	35
4.12.5 Ukuran Kotak Spasial.....	36
4.12.6 Time Travel dan Restore	37
4.13 Penyimpanan Arsip Delta	38
BAB V TUGAS PRAKTIKUM DAN PERTANYAAN EVALUASI.....	39
5.1 Persiapan Notebook Tugas	39
5.2 Rekap Benchmark R-1	40
5.3 Data Zona dalam Tiga Model R-2.....	46
5.4 Simulasi Delta Lake Dua Hari R-3.....	51

5.5 Ketentuan Pengumpulan R-4	55
5.6 Pertanyaan Evaluasi.....	55
BAB VI PENUTUP	57
6.1 Kesimpulan.....	57
DAFTAR PUSTAKA	
LAMPIRAN	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Data perjalanan taksi yang telah dibersihkan pada Praktikum 3 dapat digunakan untuk beberapa kebutuhan: pencarian detail berdasarkan ID, ringkasan pembayaran per wilayah, keterhubungan antarzona, dan audit perubahan tarif. Kebutuhan tersebut mempunyai pola akses yang berbeda. Bentuk penyimpanan yang memudahkan pencarian satu perjalanan belum tentu efisien untuk menghitung seluruh perjalanan.

Praktikum 4 melanjutkan penggunaan folder `trips_bersih/` dalam format Parquet berpartisi. Data yang sama disimpan atau direpresentasikan sebagai tabel SQLite, key-value, dokumen bersarang, penyimpanan kolom, dan graph. Pencarian spasial dipelajari menggunakan titik sintetis karena data perjalanan yang digunakan hanya menyimpan ID zona. Setiap percobaan disertai pengukuran waktu dan pemeriksaan kesesuaian hasil.

Selain memilih model data, praktikum membahas keandalan perubahan melalui Delta Lake. Penulisan, pembacaan versi lama, penolakan skema yang tidak sesuai, serta penggabungan file kecil menunjukkan hubungan antara data lake dan lakehouse. Ruang lingkup percobaan adalah tools ringan pada satu lingkungan Google Colab; sharding, replikasi, dan kinerja server database terdistribusi tidak diuji.

1.2 Tujuan Praktikum

1. Membandingkan pencarian ID dan agregasi pada SQLite serta key-value menggunakan data yang sama.
2. Menyimpan perjalanan sebagai dokumen bersarang dan mengamati field opsional serta penambahan field baru.
3. Menjalankan query DuckDB langsung pada Parquet berpartisi dan membandingkannya dengan SQLite.
4. Membangun graph arus antarzona dan memverifikasi keterhubungannya menggunakan SQL.
5. Mengukur pencarian spasial dengan loop Python, NumPy, dan STRtree pada titik sintetis.
6. Mengoperasikan versi, time travel, penegakan skema, compaction, dan restore pada Delta Lake.
7. Merancang kombinasi penyimpanan aplikasi taksi berdasarkan bukti pengukuran.

BAB II

DASAR TEORI

2.1 NoSQL dan Pemilihan Model Data

NoSQL berarti Not Only SQL: pilihan penyimpanan dapat melengkapi tabel relasional sesuai kebutuhan akses. Polyglot persistence adalah penggunaan beberapa model penyimpanan dalam satu sistem. Pada praktikum ini, pemilihan dimulai dari pertanyaan yang akan dijawab. Dict Python, TinyDB, dan NetworkX memperlihatkan model data di memori, bukan perilaku produk database terdistribusi. Penyimpanan kolom analitik Parquet juga berbeda dari wide-column store seperti Cassandra.

2.2 Tabel Relasional dan Key Value

Tabel relasional menyimpan data sebagai baris dengan kolom yang terstruktur. Indeks membantu pencarian berdasarkan nilai tertentu tanpa memindai semua baris. Key-value menghubungkan satu kunci dengan satu nilai; dict digunakan sebagai representasinya pada K-2. Nilai berupa teks JSON perlu diurai dengan `json.loads` sebelum isinya dihitung. Karena itu pencarian kunci dan agregasi mempunyai biaya yang berbeda.

2.3 Dokumen Bersarang dan Fleksibilitas Skema

Model document menyimpan satu objek sebagai dokumen yang dapat memiliki field bersarang atau opsional. Embedding menyatukan informasi terkait, seperti zona dan biaya, di dalam dokumen perjalanan sehingga dapat dibaca bersama. Referencing hanya menyimpan identitas rujukan, sementara detailnya berada pada objek lain. TinyDB dengan MemoryStorage pada K-3 menerima field cuaca baru tanpa migrasi, tetapi validasi nama dan tipe field perlu diatur pada aplikasi.

2.4 Parquet dan Query Analitik DuckDB

Parquet menyimpan data per kolom. Column pruning membatasi kolom yang dibaca, sedangkan partition pruning membatasi partisi yang relevan dengan filter. DuckDB menjalankan SQL langsung pada folder Parquet melalui sebuah view, yaitu nama query yang tidak menyalin data ke tabel baru. Perbandingan K-4 memakai seluruh data yang sama pada DuckDB dan SQLite. Selisih waktunya mencakup pengaruh engine, tata letak data, dan cara eksekusi; tidak dapat seluruhnya dianggap akibat format file.

2.5 Graph Berarah dan Keterjangkauan

Graph terdiri dari node dan edge. Pada K-5, node adalah zona dan edge berarah adalah pasangan asal-tujuan dengan bobot jumlah perjalanan. In-degree berbobot menjumlahkan perjalanan yang masuk; out-degree tanpa bobot menghitung banyak tujuan berbeda. Penelusuran tepat dua langkah berbeda makna dari penelusuran paling banyak k langkah. NetworkX digunakan untuk keduanya dan hasilnya dibandingkan dengan SQL.

2.6 Indeks Spasial STRtree

Indeks spasial mengelompokkan geometri berdasarkan kotak pembatas sehingga kelompok yang tidak berpotongan dengan wilayah pencarian dapat dilewati. STRtree pada Shapely kemudian menguji hubungan geometris melalui predicate contains. K-6 membandingkan indeks ini dengan

pemindaian loop dan operasi vektor NumPy. Waktu membangun indeks harus dibedakan dari waktu pencarian, terutama jika indeks hanya digunakan sekali.

2.7 Delta Lake dan Riwayat Transaksi

Delta Lake menambahkan transaction log pada file Parquet. Log mencatat file yang ditambahkan dan dikeluarkan dari keadaan aktif tabel. Time travel membaca keadaan pada versi tertentu; restore membuat komit baru yang mengembalikan keadaan aktif ke versi tujuan. Schema enforcement menolak penulisan yang tidak cocok. Compaction menggabungkan file aktif kecil, sementara VACUUM menghapus file lama yang memenuhi ketentuan retensi. Pada K-8 hanya dry run yang digunakan.

2.8 Posisi Praktikum dalam Pipeline Big Data

Alur praktikum dimulai dari pengukuran pada Praktikum 1, pengumpulan dan pembersihan pada Praktikum 2 sampai 3, kemudian pemilihan bentuk penyimpanan pada Praktikum 4. BASE menekankan ketersediaan dan konsistensi yang akhirnya tercapai, sedangkan ACID merangkum atomicity, consistency, isolation, dan durability dalam transaksi. CAP membahas kompromi consistency dan availability ketika terjadi gangguan komunikasi antarmesin. Sifat tersebut menjadi konteks teori; tidak diukur dengan tools satu proses pada praktikum ini.

BAB III

METODOLOGI PRAKTIKUM

3.1 Waktu dan Lingkungan Praktikum

Praktikum dikerjakan menggunakan Google Colaboratory dengan integrasi Google Drive. Metadata notebook menunjukkan kernel Python 3. Modul mengarahkan penggunaan runtime CPU, tetapi notebook tidak mencatat jenis CPU, kapasitas RAM, GPU, atau versi minor Python. Tanggal dan durasi pelaksanaan seluruh sesi tidak dicatat secara eksplisit. Simulasi hari pertama dan hari kedua pada tugas adalah tahap eksperimen, bukan dua tanggal pelaksanaan yang berbeda.

3.2 Perangkat dan Perangkat Lunak

Perangkat atau software	Versi atau spesifikasi	Kegunaan
Google Colab	Layanan daring	Menjalankan notebook
Google Drive	Layanan daring	Menyediakan data dan menyimpan keluaran
Python	Python 3; versi minor tidak dicatat	Bahasa pemrograman
pandas dan NumPy	Tidak dicatat	DataFrame, sampling, operasi vektor
PyArrow	Tidak dicatat	Dukungan pembacaan Parquet
sqlite3	Bawaan Python; versi tidak dicatat	Tabel relasional dan indeks
DuckDB	Tidak dicatat	SQL pada Parquet
TinyDB	Tidak dicatat	Dokumen di memori
NetworkX	Tidak dicatat	Graph berarah
Shapely	Tidak dicatat	Geometri dan STRtree
deltalake	1.6.6 pada log tugas dua hari	Versi dan transaksi Delta Lake

3.3 Dataset

Dataset utama adalah trips_bersih/ hasil Praktikum 3. Modul menyebut sumber perjalanan taksi New York TLC. Pada notebook, data berhasil diekstrak dari trips_bersih.zip dan dibaca dari /content/trips_bersih_p04. Output menunjukkan 2.986.910 baris dan 18 kolom sebelum penambahan id_perjalanan. Sampel acak random_state=42 berjumlah 300.000 baris dengan 19 kolom setelah ID ditambahkan. K-1b, pembangkit data utama sintetis, tidak digunakan karena dataset tersedia. File Parquet dan ZIP sumber tidak disertakan sebagai lampiran terpisah; jumlah serta struktur dilaporkan dari output notebook.

Kolom penting dan tipe pada tabel berikut mengikuti nama kolom serta skema Delta yang tercetak pada tugas. Tipe waktu_jemput menjadi teks pada sampel K-2/K-3, sedangkan pada tugas Delta tercatat timestamp_ntz. Tipe rinci seluruh 18 kolom sumber tidak dicetak sehingga tidak diasumsikan.

Kolom	Tipe yang teramati	Keterangan
waktu_jemput	timestamp_ntz; teks pada sampel	Waktu penjemputan
zona_jemput	long pada skema Delta	ID zona asal
zona_tujuan	long pada skema Delta	ID zona tujuan
wilayah_jemput	string	Wilayah; null disamakan menjadi Unknown
jarak_mil	double	Jarak perjalanan dalam mil
total_bayar	double	Jumlah pembayaran dolar AS
metode_bayar	string, nullable	Jenis pembayaran
tip	Numerik pada output; dtype tidak dicetak	Tip, dipakai pada dokumen kartu kredit
id_perjalanan	Integer dari indeks sampel + 1	ID buatan untuk pencarian; bukan ID asli TLC

Kolom lain yang tercetak adalah tpep_dropoff_datetime, passenger_count, payment_type, fare_amount, durasi_menit, jam, jam_mulai, suhu_c, hujan_mm, dan tanggal. Asal pengayaan cuaca pada Praktikum 3 tidak dijelaskan dalam lampiran yang tersedia. K-3 memakai 20.000 perjalanan pertama dari sampel dan satu dokumen tambahan buatan. K-4/K-5 memakai seluruh data. K-7/K-8 menyiapkan sampel 200.000 baris dan memakai hingga 160.000 baris. Tugas dua hari memakai sampel reservoir 100.000 baris.

Dataset spasial K-6 merupakan 200.000 titik sintetis di sekitar New York, dibangkitkan dengan seed 42. Seribu kotak pencarian dibentuk dari pusat acak. Data tersebut menguji mekanisme pencarian wilayah dan tidak membuktikan pola lokasi taksi nyata ataupun pencarian radius 500 meter.

3.4 Akses Notebook dan Repository

Tautan berikut tersimpan pada badge notebook utama:

https://colab.research.google.com/github/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/blob/main/Praktikum04/BD_A_P04_2411531001_Habibi_Putra_Rizqullah.ipynb

Repository Praktikum04:

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum04

Folder Google Drive untuk berkas pendukung:

https://drive.google.com/drive/folders/1KURZQOGDvuQaNNX6q9Af6vrRw3Zcnfmt?usp=drive_link

3.5 Tahapan Praktikum

1. K-1: memasang library, menghubungkan Drive, mengekstrak data jika diperlukan, membaca partisi, mengganti nama kolom, dan membuat sampel.
2. K-1b: jalur cadangan sintetis; tidak dipakai pada eksperimen ini.
3. K-2: membangun SQLite dan key-value, mengukur pencarian serta agregasi.
4. K-3: membangun dokumen dan mencoba field opsional maupun baru.
5. K-4: membandingkan DuckDB sampai Parquet dengan SQLite dan mencoba filter wilayah.
6. K-5: membangun graph, memeriksa arus terbesar, dan mencari keterjangkauan.
7. K-6: mengukur loop, NumPy, dan STRtree pada data spasial sintetis.
8. K-7: membuat versi Delta, melakukan update, time travel, dan uji skema.
9. K-8: melakukan append kecil, compaction, dan VACUUM dry run.
10. K-9: menyimpan benchmark dan menyusun rekap.
11. Mengerjakan analisis, studi kasus, enam latihan, tugas mandiri, pertanyaan evaluasi, serta menyiapkan arsip.

Waktu diukur memakai `time.perf_counter()`. Fungsi `ukur()` menyimpan durasi hingga empat angka desimal. Tiap pasangan pembandingan memakai cakupan data yang sama. Pengukuran yang tersimpan adalah percobaan tunggal; pengulangan, interval ketidakpastian, dan kondisi cache tidak dicatat. Rasio dihitung dari waktu yang tercetak sehingga merupakan pendekatan berdasarkan pembulatan tersebut.

BAB IV

HASIL DAN PEMBAHASAN

Kode berikut mengikuti cell notebook yang dilampirkan. Area abu-abu menampilkan input Python dengan penyorotan sintaks dan area berikutnya menampilkan output tersimpan, menyerupai pemisahan input sampai output di Colab. Nomor P adalah urutan cell kode dalam laporan; nomor cell notebook menunjukkan posisi asli, termasuk cell Markdown. Nomor eksekusi notebook utama tidak tersimpan, sehingga tidak dibuat ulang. Kode yang tersedia tanpa output ditandai tersendiri.

4.1 Menyiapkan Lingkungan dan Data Contoh K-1

A. Tujuan

Menyiapkan pustaka dan akses data, kemudian membentuk sampel yang dapat digunakan pada beberapa model penyimpanan.

B. Kode dan Output Program

Cell P1. Menghubungkan Google Drive (cell notebook 3)

```
from google.colab import drive
drive.mount('/content/drive')
```

Output tersimpan

Output tidak tersimpan pada notebook untuk cell ini.

Cell P2. Instalasi Library (cell notebook 5)

```
!pip install -q duckdb networkx shapely tinydb "deltalake>=1.0"
```

Output tersimpan

Output tidak tersimpan pada notebook untuk cell ini.

Cell P3. Menyiapkan Folder dan Ekstraksi ZIP (cell notebook 6)

```
from google.colab import drive
drive.mount("/content/drive")

import os
import zipfile

DIR_P3 = "/content/drive/MyDrive/BigData/Praktikum3"
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum4"
TRIPS_BERSIH_P4 = f"{DIR_SIMPAN}/trips_bersih"
TRIPS_BERSIH_P3 = f"{DIR_P3}/trips_bersih"
ZIP_TRIPS = f"{DIR_P3}/trips_bersih.zip"

os.makedirs(DIR_SIMPAN, exist_ok=True)

if os.path.isdir(TRIPS_BERSIH_P4):
    TRIPS_BERSIH = TRIPS_BERSIH_P4
```

```

elif os.path.isdir(TRIPS_BERSIH_P3):
    TRIPS_BERSIH = TRIPS_BERSIH_P3
else:
    TRIPS_BERSIH = TRIPS_BERSIH_P3

if not os.path.exists(TRIPS_BERSIH) and os.path.exists(ZIP_TRIPS):
    HASIL_EKSTRAK = "/content/trips_bersih_p04"
    os.makedirs(HASIL_EKSTRAK, exist_ok=True)
    with zipfile.ZipFile(ZIP_TRIPS, "r") as arsip:
        arsip.extractall(HASIL_EKSTRAK)

    kandidat = [
        os.path.join(HASIL_EKSTRAK, "trips_bersih"),
        HASIL_EKSTRAK,
    ]
    TRIPS_BERSIH = next(
        (p for p in kandidat if any(f.endswith(".parquet") for _, _, fs in os.walk(p) for f in fs)),
        TRIPS_BERSIH,
    )
    print("Dataset diekstrak dari ZIP:", TRIPS_BERSIH)

assert os.path.exists(TRIPS_BERSIH), (
    "Folder trips_bersih tidak ada. Jalankan ulang P3/K-10 atau gunakan K-1b."
)

print("Folder Praktikum 3:", DIR_P3)
print("Folder Praktikum 4:", DIR_SIMPAN)
print("Dataset:", TRIPS_BERSIH)

```

Output tersimpan

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Dataset diekstrak dari ZIP: /content/trips_bersih_p04
Folder Praktikum 3: /content/drive/MyDrive/BigData/Praktikum3
Folder Praktikum 4: /content/drive/MyDrive/BigData/Praktikum4
Dataset: /content/trips_bersih_p04

```

Cell P4. Membaca Data dan Menyiapkan Sampel (cell notebook 7)

```

import time
import json
import sqlite3
import random
import shutil
import glob

import numpy as np
import pandas as pd

catatan = []

```

```

def ukur(label, fungsi):
    t0 = time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    catatan.append({"langkah": label, "detik": round(detik, 4)})
    print(f"[{label}] {detik:.4f} s")
    return hasil

from urllib.parse import unquote

def baca_trips_berpartisi(folder):
    """Membaca file Parquet per partisi untuk menghindari ArrowInvalid.

    PyArrow kadang gagal menyatukan dictionary partisi apabila terdapat
    nilai null. Nama borough dipulihkan dari nama folder borough_naik=... .
    """
    daftar_file = sorted(
        glob.glob(os.path.join(folder, "**", "*.parquet"), recursive=True)
    )
    if not daftar_file:
        raise FileNotFoundError(f"Tidak ada file Parquet di dalam {folder}")

    bagian = []
    for path_file in daftar_file:
        potongan = pd.read_parquet(path_file)

        if "borough_naik" not in potongan.columns:
            nilai_partisi = None
            for nama_folder in path_file.split(os.sep):
                if nama_folder.startswith("borough_naik="):
                    nilai_partisi = unquote(nama_folder.split("=", 1)[1])
                    break
            potongan["borough_naik"] = (
                None if nilai_partisi == "__HIVE_DEFAULT_PARTITION__" else nilai_partisi
            )

        potongan["borough_naik"] = (
            potongan["borough_naik"].astype("string").fillna("Unknown")
        )
        bagian.append(potongan)

    return pd.concat(bagian, ignore_index=True)

df = ukur(
    "baca trips_bersih",
    lambda: baca_trips_berpartisi(TRIPS_BERSIH),
)

NAMA_BARU = {
    "tpep_pickup_datetime": "waktu_jemput",
    "PULocationID": "zona_jemput",
    "DOLocationID": "zona_tujuan",

```

```

"borough_naik": "wilayah_jemput",
"trip_distance": "jarak_mil",
"total_amount": "total_bayar",
"tip_amount": "tip",
"nama_pembayaran": "metode_bayar",
}

df = df.rename(columns=NAMA_BARU)
df["wilayah_jemput"] = df["wilayah_jemput"].astype(str)
print("baris:", f"{len(df):,}", "| kolom:", df.shape[1])

sample = df.sample(n=min(300_000, len(df)), random_state=42).reset_index(drop=True)
sample["id_perjalanan"] = sample.index + 1
sample["waktu_jemput"] = sample["waktu_jemput"].astype(str)

print("jumlah baris sampel:", f"{len(sample):,}")
print("kolom:", sample.columns.tolist())
display(sample.head())

```

Output tersimpan

```

[ baca trips_bersih ] 2.7022 s
baris: 2,986,910 | kolom: 18
jumlah baris sampel: 300,000
kolom: ['waktu_jemput', 'tpep_dropoff_datetime', 'passenger_count', 'jarak_mil',
'zona_jemput', 'zona_tujuan', 'payment_type', 'fare_amount', 'tip', 'total_bayar',
'durasi menit', 'jam', 'metode_bayar', 'jam_mulai', 'suhu_c', 'hujan_mm',
'tanggal', 'wilayah_jemput', 'id_perjalanan']

```

```

      waktu_jemput tpep_dropoff_datetime  passenger_count  jarak_mil \
0  2023-01-16 10:40:39  2023-01-16 10:52:29             1.0     1.55
1  2023-01-16 18:09:53  2023-01-16 18:16:40             2.0     0.57
2  2023-01-18 15:29:58  2023-01-18 15:44:02             1.0     1.80
3  2023-01-10 15:52:40  2023-01-10 16:14:36             1.0     3.04
4  2023-01-24 14:11:55  2023-01-24 14:35:33             1.0     3.90

      zona_jemput  zona_tujuan  payment_type  fare_amount  tip  total_bayar \
0           48         164           1      12.1  3.22      19.32
1          164         170           1       7.2  2.24      13.44
2          234         230           1      12.1  3.20      19.30
3           48         114           1      21.2  5.04      30.24
4          107         143           1      24.0  5.00      33.00

      durasi menit  jam  metode_bayar      jam_mulai  suhu_c  hujan_mm \
0  11.833333  10  Kartu kredit  2023-01-16 10:00:00    0.5    0.0
1   6.783333  18  Kartu kredit  2023-01-16 18:00:00    3.2    0.0
2  14.066667  15  Kartu kredit  2023-01-18 15:00:00   10.5    0.0
3  21.933333  15  Kartu kredit  2023-01-10 15:00:00    6.3    0.0
4  23.633333  14  Kartu kredit  2023-01-24 14:00:00    7.8    0.0

      tanggal  wilayah_jemput  id_perjalanan

```

0	2023-01-16	Manhattan	1
1	2023-01-16	Manhattan	2
2	2023-01-18	Manhattan	3
3	2023-01-10	Manhattan	4
4	2023-01-24	Manhattan	5

C. Pembahasan

Folder sumber belum tersedia langsung, sehingga notebook mengekstrak ZIP ke penyimpanan runtime. Output membuktikan pembacaan 2.986.910 baris, 18 kolom, dalam 2,7022 detik. Sampel berjumlah 300.000 baris; ID dibentuk dari indeks sampel.

Kode notebook menambahkan pembaca per file Parquet. Komentar kode menjelaskan potensi ArrowInvalid ketika dictionary partisi mengandung null. Perbaikan menjaga tujuan modul: setiap file dibaca, label borough dipulihkan dari nama folder, null menjadi Unknown, lalu bagian digabung. Tidak ada traceback error asli yang tersimpan sehingga kejadian error tidak dapat dinyatakan sebagai hasil eksekusi pada lampiran.

Jalur K-1b tidak dijalankan karena data utama tersedia. Satu pemanggilan drive.mount di awal mendahului cell instalasi; susunan ini dipertahankan sesuai notebook. Instalasi dan mount awal tidak memiliki output tersimpan.

4.2 Tabel Biasa dan Key Value K-2

A. Tujuan

Membandingkan penyimpanan dan pencarian ID serta agregasi pada sampel identik.

B. Kode dan Output Program

Cell P5. Membangun SQLite dan Key Value (cell notebook 9)

```
KOLOM = [
    "id_perjalanan", "waktu_jemput", "zona_jemput", "zona_tujuan",
    "wilayah_jemput", "jarak_mil", "total_bayar", "metode_bayar"
]
data = sample[KOLOM].copy()

if os.path.exists("uji.db"):
    os.remove("uji.db")
con = sqlite3.connect("uji.db")
ukur("relasional: muat tabel", lambda: data.to_sql("perjalanan", con, index=False))

def bangun_kv():
    return {
        f"perjalanan: {r['id_perjalanan']}": json.dumps(r)
        for r in data.to_dict("records")
    }

kv = ukur("key-value: bangun dict", bangun_kv)
print("jumlah key:", len(kv))
```

Output tersimpan

[relasional: muat tabel] 2.5168 s
[key-value: bangun dict] 9.6995 s
jumlah key: 300000

Cell P6. Mencari 200 ID Perjalanan (cell notebook 10)

```
random.seed(42)
id_uji = random.sample(range(1, len(data) + 1), 200)

def cari_sql():
    cur = con.cursor()
    return [
        cur.execute(
            "SELECT * FROM perjalanan WHERE id_perjalanan = ?", (i,)
        ).fetchone()
        for i in id_uji
    ]

def cari_kv():
    return [json.loads(kv[f"perjalanan:{i}"]) for i in id_uji]

r1 = ukur("cari 200 perjalanan - SQL tanpa indeks", cari_sql)
con.execute("CREATE INDEX idx_id ON perjalanan(id_perjalanan)")
r2 = ukur("cari 200 perjalanan - SQL dengan indeks", cari_sql)
r3 = ukur("cari 200 perjalanan - key-value", cari_kv)

print("hasil sama?", r1[0][0] == r2[0][0] == r3[0]["id_perjalanan"])
```

Output tersimpan

[cari 200 perjalanan - SQL tanpa indeks] 4.3215 s
[cari 200 perjalanan - SQL dengan indeks] 0.0037 s
[cari 200 perjalanan - key-value] 0.0012 s
hasil sama? True

Cell P7. Menghitung Pembayaran per Wilayah (cell notebook 11)

```
def hitung_sql():
    return con.execute("""
        SELECT wilayah_jemput, COUNT(*), ROUND(AVG(total_bayar), 2)
        FROM perjalanan
        GROUP BY wilayah_jemput
        ORDER BY 2 DESC, 1
    """).fetchall()

def hitung_kv():
    total, jumlah = {}, {}
    for teks in kv.values():
        d = json.loads(teks)
        w = d["wilayah_jemput"]
        total[w] = total.get(w, 0) + d["total_bayar"]
```

```

    jumlah[w] = jumlah.get(w, 0) + 1
    return sorted(
        [(w, jumlah[w], round(total[w] / jumlah[w], 2)) for w in jumlah],
        key=lambda x: (-x[1], x[0]),
    )

h1 = ukur("hitung per wilayah - SQL GROUP BY", hitung_sql)
h2 = ukur("hitung per wilayah - key-value (buka semua nilai)", hitung_kv)
print(h1[:3])
print("hasil sama?", h1 == h2)

```

Output tersimpan

```

[hitung per wilayah - SQL GROUP BY] 0.1951 s
[hitung per wilayah - key-value (buka semua nilai)] 1.4229 s
[('Manhattan', 267034, 22.7), ('Queens', 27288, 70.34), ('Unknown', 3824, 39.42
)]
hasil sama? True

```

C. Pembahasan

Pemuatan tabel memerlukan 2,5168 detik, sedangkan pembuatan dict JSON memerlukan 9,6995 detik dengan 300.000 key. Untuk 200 ID, SQL tanpa indeks membutuhkan 4,3215 detik, SQL berindeks 0,0037 detik, dan key-value 0,0012 detik. Rasio tanpa indeks terhadap berindeks adalah 1.167,97 kali; perbedaan yang besar berasal dari pemindaian tabel versus indeks.

Perbedaan SQL berindeks dan key-value sekitar 3,08 kali, tetapi selisih totalnya hanya 0,0025 detik. Ini belum membuktikan respons aplikasi produksi, karena jaringan dan server tidak diuji. Cetakan hasil sama pada pencarian hanya memeriksa ID hasil pertama, bukan seluruh isi 200 hasil; keterbatasan verifikasi tersebut perlu dibedakan dari kesetaraan agregasi.

Agregasi SQL membutuhkan 0,1951 detik dan key-value 1,4229 detik, sehingga key-value 7,29 kali lebih lambat. Setiap nilai JSON harus diurai dan dijumlahkan sendiri. Pemeriksaan `h1 == h2` menghasilkan True untuk seluruh daftar agregasi.

4.3 Document dan Fleksibilitas Data K-3

A. Tujuan

Membentuk dokumen bersarang, memeriksa field opsional, serta menambahkan field baru tanpa migrasi.

B. Kode dan Output Program

Cell P8. Membentuk Dokumen Bersarang (cell notebook 14)

```

from tinydb import TinyDB, Query
from tinydb.storages import MemoryStorage

def ke_dokumen(r):
    dok = {
        "id_perjalanan": int(r.id_perjalanan),
        "waktu_jemput": r.waktu_jemput,
        "jemput": {

```

```

        "zona": int(r.zona_jemput),
        "wilayah": r.wilayah_jemput,
    },
    "tujuan": {"zona": int(r.zona_tujuan)},
    "biaya": {
        "jarak_mil": float(r.jarak_mil),
        "total": float(r.total_bayar),
        "metode_bayar": r.metode_bayar,
    },
}
if r.metode_bayar == "Kartu kredit":
    dok["biaya"]["tip"] = float(r.tip)
return dok

db = TinyDB(storage=MemoryStorage)
tabel = db.table("perjalanan")
docs = [ke_dokumen(r) for r in sample.head(20_000).itertuples()]
tabel.insert_multiple(docs)

print("jumlah dokumen:", len(tabel))
print(json.dumps(tabel.all()[0], indent=2))

```

Output tersimpan

```

jumlah dokumen: 20000
{
  "id_perjalanan": 1,
  "waktu_jemput": "2023-01-16 10:40:39",
  "jemput": {
    "zona": 48,
    "wilayah": "Manhattan"
  },
  "tujuan": {
    "zona": 164
  },
  "biaya": {
    "jarak_mil": 1.55,
    "total": 19.32,
    "metode_bayar": "Kartu kredit",
    "tip": 3.22
  }
}

```

Cell P9. Menguji Field Opsional dan Field Baru (cell notebook 15)

```

Q = Query()

print(
    "jemput di Manhattan & total > 50 :",
    len(tabel.search((Q.jemput.wilayah == "Manhattan") & (Q.biaya.total > 50)))
,
)

```

```

print("dokumen yang punya field tip    :", len(tabel.search(Q.biaya.tip.exists
()))
print("dokumen yang TIDAK punya tip    :", len(tabel.search(~Q.biaya.tip.exist
s())))

tabel.insert({
    "id_perjalanan": 999_999,
    "waktu_jemput": "2023-01-15 08:00:00",
    "jemput": {"zona": 161, "wilayah": "Manhattan"},
    "tujuan": {"zona": 236},
    "biaya": {
        "jarak_mil": 1.2,
        "total": 14.5,
        "metode_bayar": "Kartu kredit",
        "tip": 3.0,
    },
    "cuaca": {"hujan": True, "suhu_c": -2.0},
})

print("dokumen yang punya field cuaca    :", len(tabel.search(Q.cuaca.exists()))
)
print("total dokumen sekarang            :", len(tabel))

```

Output tersimpan

```

jemput di Manhattan & total > 50 : 693
dokumen yang punya field tip    : 15906
dokumen yang TIDAK punya tip    : 4094
dokumen yang punya field cuaca    : 1
total dokumen sekarang          : 20001

```

C. Pembahasan

TinyDB menyimpan 20.000 dokumen awal. Dokumen contoh menggabungkan jemput, tujuan, dan biaya. Sebanyak 15.906 dokumen memiliki tip dan 4.094 tidak memiliki tip; keduanya berjumlah 20.000. Query penjemputan Manhattan dengan total lebih dari 50 menghasilkan 693 dokumen.

Satu dokumen buatan dengan field cuaca diterima sehingga jumlah akhir 20.001. Fleksibilitas ini memudahkan penambahan atribut, tetapi kode aplikasi tetap harus memvalidasi struktur. Keberhasilan menerima field baru bukan pengukuran kecepatan TinyDB atau bukti semua document database tidak memiliki validasi.

4.4 DuckDB pada Parquet Berpartisi K-4

A. Tujuan

Menjalankan agregasi pada seluruh data melalui DuckDB dan SQLite serta mengamati filter partisi.

B. Kode dan Output Program

Cell P10. Agregasi DuckDB pada Parquet (cell notebook 18)

```
import duckdb

GLOB = f"{TRIPS_BERSIH}/**/*.parquet"
if not glob.glob(GLOB):
    GLOB = f"{TRIPS_BERSIH}/**/*.parquet"

con_d = duckdb.connect()
con_d.sql(f"""
CREATE OR REPLACE VIEW perjalanan AS
SELECT tpep_pickup_datetime AS waktu_jemput,
       PULocationID        AS zona_jemput,
       DOLocationID        AS zona_tujuan,
       CASE WHEN borough_naik IS NULL
            OR borough_naik = '__HIVE_DEFAULT_PARTITION__'
            THEN 'Unknown' ELSE borough_naik END AS wilayah_jemput,
       trip_distance        AS jarak_mil,
       total_amount         AS total_bayar,
       nama_pembayaran      AS metode_bayar
FROM read_parquet('{GLOB}', hive_partitioning=true)
""")

SQL = """
SELECT wilayah_jemput,
       COUNT(*) AS jumlah,
       ROUND(AVG(total_bayar), 2) AS rata_bayar
FROM perjalanan
GROUP BY wilayah_jemput
ORDER BY jumlah DESC, wilayah_jemput
"""

hasil_d = ukur(
    "DuckDB - hitung per wilayah (langsung dari Parquet)",
    lambda: con_d.sql(SQL).df(),
)
display(hasil_d)
```

Output tersimpan

[DuckDB - hitung per wilayah (langsung dari Parquet)] 0.1139 s

	wilayah_jemput	jumlah	rata_bayar
0	Manhattan	2659609	22.72
1	Queens	270315	70.46
2	Unknown	37949	39.56
3	Brooklyn	15724	32.72
4	Bronx	3074	34.44
5	Staten Island	211	74.07
6	EWB	28	106.41

Cell P11. Agregasi Pembanding SQLite (cell notebook 19)

```
KOLOM_S = [
    "wilayah_jemput", "total_bayar", "jarak_mil", "zona_jemput", "zona_tujuan"
]

if os.path.exists("baris.db"):
    os.remove("baris.db")

con_s = sqlite3.connect("baris.db")
ukur(
    "SQLite - muat seluruh baris ke tabel",
    lambda: df[KOLOM_S].to_sql("perjalanan", con_s, index=False),
)

hasil_s = ukur(
    "SQLite - hitung per wilayah",
    lambda: con_s.execute("""
        SELECT wilayah_jemput, COUNT(*), ROUND(AVG(total_bayar), 2)
        FROM perjalanan
        GROUP BY wilayah_jemput
        ORDER BY 2 DESC, 1
        """).fetchall(),
)

hasil_duckdb = [tuple(r) for r in hasil_d.itertuples(index=False)]
hasil_sqlite = [tuple(r) for r in hasil_s]
print("hasil sama?", hasil_duckdb == hasil_sqlite)
```

Output tersimpan

```
[SQLite - muat seluruh baris ke tabel] 9.1240 s
[SQLite - hitung per wilayah] 1.7269 s
hasil sama? True
```

Cell P12. Filter Wilayah dan Ukuran File (cell notebook 20)

```
SQL2 = """
    SELECT COUNT(*), ROUND(AVG(total_bayar), 2)
    FROM perjalanan
    WHERE wilayah_jemput = 'Staten Island'
    """

print(
    ukur(
        "DuckDB - hanya wilayah Staten Island",
        lambda: con_d.sql(SQL2).fetchall(),
    )
)

ukuran_pq = sum(
    os.path.getsize(os.path.join(root, nama))
    for root, _, files in os.walk(TRIPS_BERSIH)
```

```

for nama in files
    if nama.endswith(".parquet")
) / 1024**2

ukuran_sqlite = os.path.getsize("baris.db") / 1024**2
print(
    f"Parquet ({df.shape[1]} kolom): {ukuran_pq:.1f} MB | "
    f"SQLite ({len(KOLOM_S)} kolom): {ukuran_sqlite:.1f} MB"
)

```

Output tersimpan

```

[DuckDB - hanya wilayah Staten Island] 0.0176 s
[(211, 74.07)]
Parquet (18 kolom): 66.5 MB | SQLite (5 kolom): 114.2 MB

```

C. Pembahasan

DuckDB menghasilkan agregasi 2.986.910 baris dalam 0,1139 detik dan SQLite dalam 1,7269 detik, dengan hasil sama True. DuckDB 15,16 kali lebih cepat untuk query ini. SQLite juga memerlukan 9,1240 detik untuk memuat data; waktu muat dipisahkan dari waktu query.

Query hanya membutuhkan wilayah_jemput dan total_bayar, sehingga penyimpanan kolom memungkinkan column pruning. Pemrosesan vektor pada engine juga dapat memengaruhi durasi. Jumlah thread dan profil eksekusi tidak dicatat, sehingga pemakaian beberapa inti pada sesi ini tidak dapat dipastikan hanya dari waktu tersebut.

Filter Staten Island menghasilkan 211 baris dengan rata-rata 74,07 dalam 0,0176 detik, sekitar 6,47 kali lebih cepat dari agregasi penuh. Hasil sesuai manfaat pruning, tetapi tidak ada EXPLAIN yang memastikan partisi mana yang benar-benar dibaca. View notebook memakai CASE untuk normalisasi Unknown, berbeda dari alias langsung pada modul.

Ukuran Parquet tercetak 66,5 MB untuk 18 kolom, sedangkan SQLite 114,2 MB untuk lima kolom. Ini merupakan hasil ukuran artefak yang berbeda struktur, bukan perbandingan rasio kompresi pada skema identik.

4.5 Graph dan Keterhubungan Antarzona K-5

A. Tujuan

Meringkas perjalanan menjadi arus berarah dan memeriksa keterjangkauan dengan graph serta SQL.

B. Kode dan Output Program

Cell P13. Membangun Graph Arus Antarzona (cell notebook 23)

```

import networkx as nx

arus = ukur(
    "hitung arus antar zona",
    lambda: con_d.sql("""
        SELECT zona_jemput AS asal,
               zona_tujuan AS tujuan,

```

```

        COUNT(*) AS jumlah
    FROM perjalanan
    GROUP BY 1, 2
    """).df(),
)

G = nx.DiGraph()
for r in arus.itertuples(index=False):
    G.add_edge(int(r.asal), int(r.tujuan), jumlah=int(r.jumlah))

print("node (zona):", G.number_of_nodes(), "| edge (arus):", G.number_of_edges())

top_graf = sorted(
    G.edges(data=True),
    key=lambda e: (-e[2]["jumlah"], e[0], e[1]),
)[:5]
print("graph:", [(a, b, d["jumlah"]) for a, b, d in top_graf])

top_sql = arus.sort_values(
    ["jumlah", "asal", "tujuan"], ascending=[False, True, True]
).head(5)
print("SQL :", list(top_sql.itertuples(index=False, name=None)))

populer = sorted(
    G.in_degree(weight="jumlah"), key=lambda kv: (-kv[1], kv[0])
)[:5]
print("zona tujuan terpopuler:", populer)

```

Output tersimpan

```

[hitung arus antar zona] 0.2170 s
node (zona): 262 | edge (arus): 21583
graph: [(237, 236, 22065), (236, 237, 18810), (236, 236, 14110), (237, 237, 13753), (264, 264, 13562)]
SQL : [(237, 236, 22065), (236, 237, 18810), (236, 236, 14110), (237, 237, 13753), (264, 264, 13562)]
zona tujuan terpopuler: [(236, 144209), (237, 130369), (161, 113991), (230, 87706), (170, 87110)]

```

Cell P14. Keterjangkauan Graph dan SQL (cell notebook 24)

```

MIN = 50
zona_awal = populer[0][0]

def dua_langkah_graph():
    hasil = set()
    for b, d1 in G[zona_awal].items():
        if d1["jumlah"] < MIN:
            continue
        for c, d2 in G[b].items():
            if d2["jumlah"] >= MIN and c != zona_awal:
                hasil.add(c)

```

```

return hasil

con_d.register("arus_df", arus)

def dua_langkah_sql():
    return set(con_d.sql(f"""
        SELECT DISTINCT a2.tujuan
        FROM arus_df a1
        JOIN arus_df a2 ON a1.tujuan = a2.asal
        WHERE a1.asal = {zona_awal}
        AND a1.jumlah >= {MIN}
        AND a2.jumlah >= {MIN}
        AND a2.tujuan <> {zona_awal}
        """).df()["tujuan"])

hg = ukur("2 langkah - graph (NetworkX)", dua_langkah_graph)
hs = ukur("2 langkah - SQL self-join (DuckDB)", dua_langkah_sql)

print(
    "zona awal:", zona_awal,
    "| terjangkau (graph):", len(hg),
    "| (SQL):", len(hs),
    "| sama?", hg == hs,
)

H = nx.DiGraph([
    (a, b) for a, b, d in G.edges(data=True) if d["jumlah"] >= MIN
])

for k in (1, 2, 3, 4):
    n = len(nx.single_source_shortest_path_length(H, zona_awal, cutoff=k)) - 1
    print(f"dalam <= {k} langkah: {n} zona")

```

Output tersimpan

```

[2 langkah - graph (NetworkX)] 0.0068 s
[2 langkah - SQL self-join (DuckDB)] 0.0081 s
zona awal: 236 | terjangkau (graph): 214 | (SQL): 214 | sama? True
dalam <= 1 langkah: 74 zona
dalam <= 2 langkah: 214 zona
dalam <= 3 langkah: 214 zona
dalam <= 4 langkah: 214 zona

```

C. Pembahasan

Graph memiliki 262 node dan 21.583 edge; agregasi arus memerlukan 0,2170 detik. Lima arus terbesar sama antara graph dan SQL. Zona tujuan terpopuler adalah 236 dengan 144.209 perjalanan masuk. Output juga memuat ID 264, sehingga nilai ID aktual tidak dipaksa mengikuti rentang 1 sampai 263 pada uraian umum modul.

Dengan ambang minimal 50 perjalanan pada setiap edge, penelusuran tepat dua langkah dari zona 236 menghasilkan 214 zona pada graph dan SQL. Waktunya 0,0068 dan 0,0081 detik,

selisih 0,0013 detik. Sementara itu penelusuran paling banyak langkah menghasilkan 74, 214, 214, dan 214 zona untuk k=1 sampai 4. Kesamaan jumlah pada dua langkah tidak membuat definisi tepat dua dan paling banyak dua menjadi identik secara umum.

Graph memudahkan perubahan cutoff tanpa menyusun rangkaian self-join secara manual. Hasil menunjukkan jangkauan stabil sejak k=2 pada graph yang telah disaring, bukan semua zona dataset terjangkau. SQL juga dapat memakai recursive CTE; jadi kebutuhan menambah JOIN berlaku pada formulasi eksplisit yang digunakan dalam percobaan.

4.6 Indeks Spasial K-6

A. Tujuan

Membandingkan pencarian wilayah melalui loop, NumPy, dan STRtree pada titik sintetis.

B. Kode dan Output Program

Cell P15. Pencarian Kotak dengan Tiga Cara (cell notebook 27)

```
from shapely import STRtree, box, points

rng = np.random.default_rng(42)
N = 200_000
bujur = rng.normal(-73.97, 0.06, N).clip(-74.25, -73.70)
lintang = rng.normal(40.74, 0.05, N).clip(40.50, 40.92)
titik = points(bujur, lintang)

M = 1000
cx = rng.uniform(-74.05, -73.85, M)
cy = rng.uniform(40.65, 40.85, M)
kotak = [
    box(x - 0.005, y - 0.005, x + 0.005, y + 0.005)
    for x, y in zip(cx, cy)
]

def cara_loop():
    return [sum(1 for p in titik if k.contains(p)) for k in kotak[:5]]

def cara_numpy():
    hasil = []
    for x, y in zip(cx, cy):
        mask = (
            (bujur >= x - 0.005)
            & (bujur <= x + 0.005)
            & (lintang >= y - 0.005)
            & (lintang <= y + 0.005)
        )
        hasil.append(int(mask.sum()))
    return hasil

h_loop = ukur("cara 1 - loop Python, 5 kotak", cara_loop)
h_np = ukur("cara 2 - NumPy, 1000 kotak", cara_numpy)

pohon = ukur("bangun indeks STRtree", lambda: STRtree(titik))
h_idx = ukur(
```

```

"car 3 - STRtree, 1000 kotak",
lambda: [len(pohon.query(k, predicate="contains")) for k in kotak],
)

print("5 kotak pertama - loop:", h_loop, "| STRtree:", h_idx[:5])
print("hasil NumPy dan STRtree sama?", h_np == h_idx)
print("loop dan STRtree sama (5 pertama)?", h_loop == h_idx[:5])
print("total titik ditemukan (1000 kotak):", sum(h_idx))

```

Output tersimpan

```

[cara 1 - loop Python, 5 kotak] 13.9702 s
[cara 2 - NumPy, 1000 kotak] 0.5210 s
[bangun indeks STRtree] 0.0788 s
[cara 3 - STRtree, 1000 kotak] 0.2139 s
5 kotak pertama - loop: [1012, 84, 228, 748, 509] | STRtree: [1012, 84, 228, 74
8, 509]
hasil NumPy dan STRtree sama? True
loop dan STRtree sama (5 pertama)? True
total titik ditemukan (1000 kotak): 417545

```

C. Pembahasan

Loop Python menguji lima kotak dalam 13,9702 detik. NumPy menguji 1.000 kotak dalam 0,5210 detik. Pembuatan STRtree memerlukan 0,0788 detik, dan 1.000 pencariannya 0,2139 detik. Total biaya membangun dan mencari adalah 0,2927 detik; percepatan terhadap NumPy 1,78 kali, atau 2,44 kali jika indeks sudah tersedia.

Ekstrapolasi linier loop ke 1.000 kotak adalah $13,9702/5 \times 1.000 = 2.794,04$ detik. Rasio sekitar 9.545,06 kali terhadap total STRtree merupakan perkiraan, bukan hasil loop 1.000 kotak yang dijalankan. Waktu rata-rata loop dan STRtree juga berasal dari jumlah serta cakupan kotak berbeda sehingga rasio itu tidak boleh dianggap pengukuran identik.

Hasil lima kotak pertama sama, dan seluruh hasil NumPy sampai STRtree sama. Total 417.545 adalah penjumlahan temuan per kotak; satu titik dapat ditemukan pada lebih dari satu kotak sehingga bukan jumlah titik unik. NumPy memakai batas inklusif, sedangkan contains tidak memasukkan batas geometri; kesamaan yang tercetak berlaku pada titik uji ini.

4.7 Delta Lake Versi Time Travel dan Skema K-7

A. Tujuan

Menguji riwayat versi, koreksi pembayaran, pembacaan versi lama, dan penolakan kolom yang tidak sesuai.

B. Kode dan Output Program

Cell P16. Menulis dan Memperbarui Tabel Delta (cell notebook 30)

```

from deltalake import DeltaTable, write_deltalake

KOLOM_DELTA = [
    "waktu_jemput", "zona_jemput", "zona_tujuan",
    "wilayah_jemput", "jarak_mil", "total_bayar", "metode_bayar"
]

```

```

]

data_delta = df[KOLOM_DELTA].sample(
    n=min(200_000, len(df)), random_state=42
).reset_index(drop=True)

PATH_DELTA = os.path.join(os.getcwd(), "trips_delta")
if os.path.exists(PATH_DELTA):
    shutil.rmtree(PATH_DELTA)

write_deltalake(PATH_DELTA, data_delta.iloc[:100_000], mode="overwrite")
dt = DeltaTable(PATH_DELTA)
print("versi:", dt.version(), "| baris:", len(dt.to_pandas()))

write_deltalake(PATH_DELTA, data_delta.iloc[100_000:150_000], mode="append")
dt = DeltaTable(PATH_DELTA)
print("versi:", dt.version(), "| baris:", len(dt.to_pandas()))

sekarang = data_delta.iloc[:150_000]
n_koreksi = int((
    (sekarang["wilayah_jemput"] == "Bronx")
    & (sekarang["metode_bayar"] == "Tunai")
).sum())
print("baris yang akan dikoreksi:", n_koreksi)

dt.update(
    predicate="wilayah_jemput = 'Bronx' AND metode_bayar = 'Tunai'",
    updates={"total_bayar": "total_bayar + 1.0"},
)
print("versi:", DeltaTable(PATH_DELTA).version())

```

Output tersimpan

```

versi: 0 | baris: 100000
versi: 1 | baris: 150000
baris yang akan dikoreksi: 20
versi: 2

```

Cell P17. Membaca Versi Lama dan Riwayat (cell notebook 31)

```

v0 = DeltaTable(PATH_DELTA, version=0).to_pandas()
v1 = DeltaTable(PATH_DELTA, version=1).to_pandas()
terbaru = DeltaTable(PATH_DELTA).to_pandas()

print("baris v0:", len(v0), "| v1:", len(v1), "| terbaru:", len(terbaru))
selisih = terbaru["total_bayar"].sum() - v1["total_bayar"].sum()
print(
    "selisih total_bayar (terbaru - v1):",
    round(selisih, 2),
    "| jumlah baris dikoreksi:",
    n_koreksi,
)

```

```
print("\nRiwayat perubahan:")
for h in DeltaTable(PATH_DELTA).history():
    print(h.get("version"), h.get("operation"))

print("\nIsi _delta_log:")
print(sorted(os.listdir(f"{PATH_DELTA}/_delta_log"))))
```

Output tersimpan

```
baris v0: 100000 | v1: 150000 | terbaru: 150000
selisih total_bayar (terbaru - v1): 20.0 | jumlah baris dikoreksi: 20
```

Riwayat perubahan:

2 UPDATE

1 WRITE

0 WRITE

Isi_delta_log:

[illegible]

Cell P18. Menguji Penolakan Kolom Asing (cell notebook 32)

```
tambahan = data_delta.iloc[150_000:150_010].copy()
tambahan["kolom_asing"] = 1

try:
    write_deltalake(PATH_DELTA, tambahan, mode="append")
    print("append kolom asing: BERHASIL (tidak diharapkan)")
except Exception as e:
    print("append kolom asing DITOLAK:", type(e).__name__)

print("versi setelah percobaan gagal:", DeltaTable(PATH_DELTA).version())
```

Output tersimpan

append kolom asing DITOLAK: SchemaMismatchError
versi setelah percobaan gagal: 2

C. Pembahasan

Versi 0 memuat 100.000 baris, versi 1 memuat 150.000 baris, dan update membuat versi 2 dengan jumlah baris tetap. Koreksi 20 perjalanan Tunai di Bronx sebesar 1,0 menghasilkan selisih total_bayar 20,0. Versi 0 dan 1 tetap dapat dibaca sehingga perubahan tidak menghapus keadaan lama dari riwayat log.

Penambahan kolom_asing ditolak dengan `SchemaMismatchError` dan versi tetap 2. Ini membuktikan tidak ada komit tabel baru yang teramati setelah penolakan. Output tersebut tidak memeriksa seluruh file fisik, sehingga tidak dipakai untuk mengklaim bahwa tidak mungkin ada artefak yatim pada setiap jenis kegagalan.

PATH_DELTA notebook berada di direktori runtime melalui `os.getcwd()`, berbeda dari lokasi Drive pada modul. Tujuan percobaan tetap sama dan hasil akhirnya diekspor sebagai ZIP ke Drive.

4.8 Small File Problem dan Compaction K-8

A. Tujuan

Meniru penambahan kecil berulang dan mengamati perubahan file aktif, waktu baca, serta file lama.

B. Kode dan Output Program

Cell P19. Append Kecil dan Compaction (cell notebook 35)

```
dt = DeltaTable(PATH_DELTA)
print("file data aktif sebelum:", len(dt.file_uris()), "| versi:", dt.version()
)

for i in range(20):
    awal = 150_000 + i * 500
    akhir = 150_000 + (i + 1) * 500
    potongan = data_delta.iloc[awal:akhir]
    write_deltalake(PATH_DELTA, potongan, mode="append")

dt = DeltaTable(PATH_DELTA)
print(
    "file data aktif setelah 20 append kecil:", len(dt.file_uris()),
    "| versi:", dt.version(),
    "| baris:", len(dt.to_pandas()),
)

t0 = time.perf_counter()
_ = DeltaTable(PATH_DELTA).to_pandas()
t_sebelum = time.perf_counter() - t0

hasil = dt.optimize.compact()
print(
    "hasil compact:",
    {k: hasil[k] for k in ("numFilesAdded", "numFilesRemoved")},
)

dt = DeltaTable(PATH_DELTA)
t0 = time.perf_counter()
_ = DeltaTable(PATH_DELTA).to_pandas()
t_sesudah = time.perf_counter() - t0

print(
    "file data aktif setelah compact:", len(dt.file_uris()),
    "| versi:", dt.version(),
    "| baris:", len(dt.to_pandas()),
)
print(
    f"waktu baca sebelum compact: {t_sebelum:.4f} s | "
    f"sesudah: {t_sesudah:.4f} s"
```

```
)

print("file parquet di disk:", len(glob.glob(f"{PATH_DELTA}/*.parquet")))
kandidat = dt.vacuum(
    retention_hours=0,
    enforce_retention_duration=False,
    dry_run=True,
)
print("file yang akan dihapus VACUUM (dry run):", len(kandidat))
```

Output tersimpan

```
file data aktif sebelum: 1 | versi: 2
file data aktif setelah 20 append kecil: 21 | versi: 22 | baris: 160000
hasil compact: {'numFilesAdded': 1, 'numFilesRemoved': 21}
file data aktif setelah compact: 1 | versi: 23 | baris: 160000
waktu baca sebelum compact: 0.0780 s | sesudah: 0.0525 s
file parquet di disk: 24
file yang akan dihapus VACUUM (dry run): 23
```

C. Pembahasan

Dua puluh append masing-masing 500 baris menaikkan jumlah data dari 150.000 menjadi 160.000 dan file aktif dari satu menjadi 21 pada versi 22. Compaction menghasilkan versi 23, menambahkan satu file dan mengeluarkan 21 file lama dari keadaan aktif. Jumlah baris tetap 160.000.

Waktu baca turun dari 0,0780 menjadi 0,0525 detik, sekitar 1,49 kali pada pengukuran ini. Jumlah file Parquet fisik masih 24 karena log mempertahankan file versi sebelumnya. VACUUM dry run melaporkan 23 kandidat dan tidak menghapusnya. Retention nol pada dry run memperlihatkan cakupan penghapusan; pelaksanaan nyata dapat memutus time travel dan mengganggu pembaca lama.

4.9 Rekap Benchmark K-9

A. Tujuan

Menyimpan waktu eksperimen dan merangkum kecocokan model berdasarkan kebutuhan.

B. Kode dan Output Program

Cell P20. Menyimpan Benchmark (cell notebook 38)

```
rekap = pd.DataFrame(catatan)
PATH_BENCHMARK = f"{DIR_SIMPAN}/benchmark_nosql.csv"
rekap.to_csv(PATH_BENCHMARK, index=False)

print(rekap.to_string(index=False))
print("\nBenchmark tersimpan di:", PATH_BENCHMARK)
```

Output tersimpan

```
langkah    detik
baca trips_bersih  2.7022
```

```

relasional: muat tabel 2.5168
key-value: bangun dict 9.6995
cari 200 perjalanan - SQL tanpa indeks 4.3215
cari 200 perjalanan - SQL dengan indeks 0.0037
cari 200 perjalanan - key-value 0.0012
hitung per wilayah - SQL GROUP BY 0.1951
hitung per wilayah - key-value (buka semua nilai) 1.4229
DuckDB - hitung per wilayah (langsung dari Parquet) 0.1139
SQLite - muat seluruh baris ke tabel 9.1240
SQLite - hitung per wilayah 1.7269
DuckDB - hanya wilayah Staten Island 0.0176
hitung arus antar zona 0.2170
2 langkah - graph (NetworkX) 0.0068
2 langkah - SQL self-join (DuckDB) 0.0081
cara 1 - loop Python, 5 kotak 13.9702
cara 2 - NumPy, 1000 kotak 0.5210
bangun indeks STRtree 0.0788
cara 3 - STRtree, 1000 kotak 0.2139

```

Benchmark tersimpan di: /content/drive/MyDrive/BigData/Praktikum4/benchmark_nosql.csv

Cell P21. Menghitung Rasio Waktu (cell notebook 39)

```

waktu = rekap.set_index("langkah")["detik"]

def rasio(pembilang, penyebut):
    return waktu[pembilang] / waktu[penyebut]

ringkasan_rasio = pd.DataFrame({
    "perbandingan": [
        "SQL tanpa indeks / SQL berindeks",
        "SQL berindeks / key-value",
        "key-value agregasi / SQL GROUP BY",
        "SQLite agregasi / DuckDB agregasi",
        "SQL dua langkah / graph dua langkah",
        "NumPy spasial / STRtree spasial",
    ],
    "rasio": [
        rasio("cari 200 perjalanan - SQL tanpa indeks", "cari 200 perjalanan - SQL dengan indeks"),
        rasio("cari 200 perjalanan - SQL dengan indeks", "cari 200 perjalanan - key-value"),
        rasio("hitung per wilayah - key-value (buka semua nilai)", "hitung per wilayah - SQL GROUP BY"),
        rasio("SQLite - hitung per wilayah", "DuckDB - hitung per wilayah (langsung dari Parquet)"),
        rasio("2 langkah - SQL self-join (DuckDB)", "2 langkah - graph (NetworkX)"),
        rasio("cara 2 - NumPy, 1000 kotak", "cara 3 - STRtree, 1000 kotak"),
    ],
})

```

```
ringkasan_rasio["rasio"] = ringkasan_rasio["rasio"].round(2)
display(ringkasan_rasio)
```

Output tersimpan

```
perbandingan  rasio
0  SQL tanpa indeks / SQL berindeks  1167.97
1      SQL berindeks / key-value      3.08
2  key-value agregasi / SQL GROUP BY   7.29
3  SQLite agregasi / DuckDB agregasi   15.16
4  SQL dua langkah / graph dua langkah  1.19
5  NumPy spasial / STRtree spasial     2.44
```

C. Pembahasan

Notebook mencetak 19 baris benchmark dan path benchmark_nosql.csv di folder Praktikum4. Rasio utama pada cell berikutnya sesuai dengan waktu tersimpan. Catatan ini berhenti pada K-9: waktu Latihan 3 yang ditambahkan sesudahnya tidak otomatis masuk ke CSV yang telah disimpan, dan waktu baca Delta K-8 diukur terpisah dari fungsi ukur().

Rekap kebutuhan lengkap dengan satu interpretasi per kebutuhan dibahas pada Tugas R-1. Pencarian ID dan agregasi key-value dibandingkan dengan SQL pada 300.000 baris, sedangkan DuckDB dibandingkan dengan SQLite pada 2.986.910 baris. Angka kedua kelompok tidak digunakan sebagai pembandingan langsung antar model dengan cakupan berbeda.

4.10 Analisis Hasil

1. Pencarian satu data

SQL tanpa indeks 1.167,97 kali lebih lambat daripada SQL berindeks. SQL berindeks 3,08 kali lebih lambat daripada key-value, dengan selisih hanya 2,5 milidetik untuk 200 ID. Indeks memberi perubahan terbesar karena menghindari pemindaian berulang.

2. Harga kesederhanaan key-value

Agregasi key-value 7,29 kali lebih lambat daripada SQL GROUP BY, karena 300.000 nilai JSON harus diurai sebelum atribut pembayaran dapat dijumlahkan.

3. Fleksibilitas dokumen

Field cuaca dapat diterima pada dokumen ke-20.001. Keuntungannya adalah penambahan atribut tanpa migrasi; risikonya nama atau tipe field yang tidak konsisten perlu diperiksa aplikasi.

4. Penyimpanan per kolom

DuckDB 15,16 kali lebih cepat dari SQLite untuk query yang sama pada seluruh dataset. Column pruning adalah penjelasan terkait format; pemrosesan vektor engine adalah penjelasan selain format. Kontribusi setiap penyebab belum diukur secara terpisah.

5. Kelayakan graph

Graph dan SQL menghasilkan 214 zona tepat dua langkah dengan durasi mendekati. Hasil k=1 sampai 4, yaitu 74, 214, 214, 214, menunjukkan kemudahan memperluas jumlah langkah dan membaca struktur jaringan melalui API graph.

6. Klaim percepatan spasial

STRtree menghabiskan 0,2927 detik termasuk pembangunan indeks untuk 1.000 kotak, dibanding NumPy 0,5210 detik atau 1,78 kali. Rasio terhadap ekstrapolasi loop sekitar 9.545,06 kali bukan pengukuran loop penuh. Pembandingan, ukuran wilayah, dan biaya membangun indeks menentukan makna klaim.

7. Delta Lake

Versi 1 masih dapat dibaca dengan 150.000 baris setelah koreksi versi 2, dan selisih pembayaran sebesar 20,0 sesuai 20 baris yang diubah. Penolakan skema tidak menambah versi. Setelah compaction, satu file aktif dan 24 file fisik berbeda karena file lama dipertahankan untuk riwayat.

4.11 Studi Kasus Rancangan Penyimpanan Aplikasi Taksi

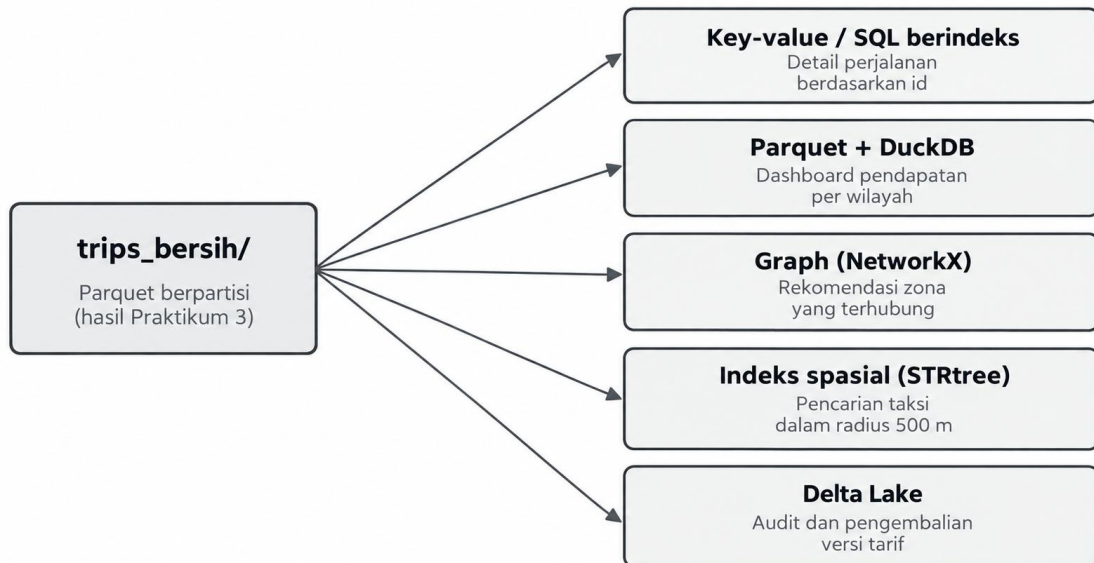
Rancangan berikut menjawab lima fitur pada Bagian P modul. Pilihan penyimpanan mengacu pada hasil K-2 sampai K-8, sedangkan persyaratan aplikasi adalah sasaran rancangan. Pengukuran Colab tidak mencakup latensi jaringan, beban banyak pengguna, atau radius geodesik 500 meter.

Fitur	Pilihan dan alasan hasil ukur	Kelemahan yang diwaspadai
Detail perjalanan	Key-value; SQLite berindeks sebagai alternatif. 200 ID: 0,0012 s dan 0,0037 s; tanpa indeks 4,3215 s.	Biaya membangun dict 9,6995 s; bukan ukuran latensi server produksi.
Dashboard pendapatan	Parquet + DuckDB untuk analitik. Agregasi per wilayah 0,1139 s versus SQLite 1,7269 s; filter Staten Island 0,0176 s.	Pengelompokan per jam dan filter tanggal perlu query tambahan; belum diukur pada K-4.
Rekomendasi zona	NetworkX graph. Tepat dua langkah: 0,0068 s, sama dengan SQL pada 214 zona. Cutoff dapat diganti.	Graph ringkasan perlu diperbarui ketika sumber berubah.
Radius 500 meter	Indeks spasial STRtree sebagai kandidat. 1.000 kotak: query 0,2139 s dan pembangunan 0,0788 s.	Uji memakai titik sintetis dan kotak derajat; koordinat nyata serta pengujian jarak diperlukan.
Audit tarif	Delta Lake. Versi lama dapat dibaca; update 20 baris memberi selisih 20,0; restore menghasilkan versi baru.	Time travel bergantung pada file riwayat yang belum dihapus; retensi harus sesuai audit.

Penyimpanan relasional berindeks tetap layak jika kebutuhan transaksi dan konsistensi lebih dominan. Hasil pencarian cepat pada dict menunjukkan kesesuaian pola kunci, bukan alasan tunggal mengganti database aplikasi. Untuk dashboard, data per jam sudah tersedia pada dataset, tetapi query pendapatan per jam belum menjadi bagian pengukuran ini.

4.11.1 Aliran Data dan Kebutuhan Lokasi

Aliran data dari trips_bersih/ ke tiap penyimpanan



Gambar 4.1 Diagram yang tersimpan pada output notebook utama.

Diagram memperlihatkan target rancangan, bukan bukti bahwa semua fitur produksi sudah diterapkan. Cabang spasial memerlukan data koordinat tambahan. Pada eksperimen K-6, titik dibangkitkan secara sintetis secara terpisah dari trips_bersih/, sehingga belum menjadi aliran lokasi nyata dari dataset perjalanan.

Untuk radius 500 meter, rancangan perlu sistem koordinat dan perhitungan jarak yang sesuai. Indeks dapat menyaring kandidat dengan kotak pembatas terlebih dahulu, kemudian setiap kandidat diperiksa berdasarkan jarak sebenarnya. Menyamakan 0,005 derajat dengan 500 meter secara tetap tidak tepat karena ukuran derajat bujur bergantung pada lintang. Waktu uji kotak tidak dapat dilaporkan sebagai waktu query radius.

Node dan edge graph menyimpan ringkasan pasangan zona; data pembayaran lengkap tetap berada di sumber. Karena itu query rata-rata tarif tidak dapat dijawab dari bobot graph yang hanya berisi jumlah perjalanan. Data koordinat dan identitas taksi juga harus tersedia sebelum fitur pencarian taksi berdasarkan lokasi dapat dijalankan.

4.11.2 Data Ganda dan Sinkronisasi

Rancangan menyimpan representasi ganda: salinan JSON untuk detail perjalanan, agregat antarzona untuk graph, dan representasi kolom untuk analitik. Tabel Delta dapat menjadi sumber berversi yang digunakan untuk audit dan pembaruan terkontrol. Diagram memakai trips_bersih/ sebagai bahan awal; penempatan Delta sebagai sumber pembaruan adalah usulan rancangan, bukan konfigurasi layanan yang telah diimplementasikan.

Pembaruan sebaiknya dilakukan pada satu sumber berversi. Setelah komit berhasil, proses turunan membangun kembali key-value dan ringkasan graph dari versi yang sama, lalu mencatat versi sumber dan waktu pembaruan. Pemeriksaan jumlah baris, kunci, serta hasil agregasi membantu mendeteksi salinan yang tidak sesuai. Peralihan ke salinan baru perlu dilakukan setelah proses pembentukan dan pemeriksaan selesai.

Sinkronisasi tersebut belum dijalankan sebagai pipeline otomatis pada notebook. K-2, K-4, dan K-5 hanya menunjukkan sebagian pemeriksaan kesesuaian query. Untuk detail perjalanan, ID yang dibentuk ulang dari indeks sampel perlu diganti dengan identitas stabil pada rancangan aplikasi; jika sampling berubah, key yang sama belum tentu menunjuk perjalanan yang sama.

Dashboard dan rekomendasi dapat menerima keterlambatan yang disepakati, sementara audit membaca versi Delta secara langsung. Retensi file harus diselaraskan dengan periode audit, karena restore dan time travel tidak dapat mengembalikan file yang telah dibuang oleh VACUUM. Dengan demikian, keputusan penggunaan beberapa model disertai biaya pemeliharaan salinan dan riwayat, bukan hanya keuntungan waktu query.

4.12 Latihan Praktikum

4.12.1 Pencarian 2000 ID

Cell P22. Pencarian 2000 ID (cell notebook 45)

```
random.seed(42)
id_uji_2000 = random.sample(range(1, len(data) + 1), 2_000)

def cari_sql_2000():
    cur = con.cursor()
    return [
        cur.execute(
            "SELECT * FROM perjalanan WHERE id_perjalanan = ?", (i,)
        ).fetchone()
        for i in id_uji_2000
    ]

def cari_kv_2000():
    return [json.loads(kv[f"perjalanan:{i}"]) for i in id_uji_2000]

t0 = time.perf_counter()
_ = cari_sql_2000()
waktu_sql_2000 = time.perf_counter() - t0

t0 = time.perf_counter()
_ = cari_kv_2000()
waktu_kv_2000 = time.perf_counter() - t0

print(f"SQL berindeks: {waktu_sql_2000:.6f} s | {waktu_sql_2000 / 2000 * 1e6:.2f} μs/pencarian")
print(f"Key-value : {waktu_kv_2000:.6f} s | {waktu_kv_2000 / 2000 * 1e6:.2f} μs/pencarian")
```

Output tersimpan

SQL berindeks: 0.040489 s | 20.24 μs/pencarian
Key-value : 0.013244 s | 6.62 μs/pencarian

Pembahasan

SQL berindeks memerlukan 0,040489 detik atau 20,24 mikrodetik per pencarian; key-value memerlukan 0,013244 detik atau 6,62 mikrodetik. Nilai per pencarian dihitung dari durasi total dibagi 2.000. Selisih sekitar 13,62 mikrodetik merupakan pengukuran lokal, bukan latensi permintaan aplikasi.

4.12.2 Query Document dan SQL

Cell P23. Query Document dan SQL (cell notebook 47)

```
zona_uji = int(sample.iloc[0]["zona_tujuan"])

jumlah_doc = len(tabel.search(
    (Q.tujuan.zona == zona_uji)
```

```

    & (Q.biaya.metode_bayar == "Tunai")
))

jumlah_sql = con.execute("""
    SELECT COUNT(*)
    FROM perjalanan
    WHERE zona_tujuan = ? AND metode_bayar = 'Tunai'
""", (zona_uji,)).fetchone()[0]

print("zona tujuan:", zona_uji)
print("document (20.000 baris):", jumlah_doc)
print("SQL (300.000 baris)   :", jumlah_sql)
print("sama?", jumlah_doc == jumlah_sql)
print("Catatan: cakupan data berbeda sehingga hasil jumlah dapat berbeda.")

```

Output tersimpan

```

zona tujuan: 164
document (20.000 baris): 93
SQL (300.000 baris)   : 1086
sama? False
Catatan: cakupan data berbeda sehingga hasil jumlah dapat berbeda.

```

Cell P24. Verifikasi SQL pada Cakupan 20000 Baris (cell notebook 48)

```

jumlah_sql_20k = con.execute("""
    SELECT COUNT(*)
    FROM perjalanan
    WHERE id_perjalanan <= 20000
    AND zona_tujuan = ? AND metode_bayar = 'Tunai'
""", (zona_uji,)).fetchone()[0]

print("document (20.000 dokumen)   :", jumlah_doc)
print("SQL (20.000 baris pertama) :", jumlah_sql_20k)
print("sama?", jumlah_doc == jumlah_sql_20k)

```

Output tersimpan

Output tidak tersimpan pada notebook untuk cell ini.

Pembahasan

TinyDB menghasilkan 93 dokumen dan SQLite 1.086 baris untuk zona tujuan 164 serta pembayaran Tunai. Cakupannya berbeda: 20.000 dokumen versus 300.000 baris. Cell pembatas `id_perjalanan <= 20000` telah ditulis tetapi tidak memiliki output tersimpan; kesamaan pada cakupan tersebut belum dibuktikan pada lampiran. Dokumen tambahan tidak memenuhi kondisi query.

4.12.3 SUM Satu dan Beberapa Kolom

Cell P25. SUM Satu dan Beberapa Kolom (cell notebook 51)

```

q_satu = "SELECT SUM(total_bayar) FROM perjalanan"

```

```

q_banyak = """
    SELECT SUM(total_bayar), SUM(jarak_mil),
           SUM(zona_jemput), SUM(zona_tujuan)
    FROM perjalanan
"""

t_satu = ukur("latihan: SUM satu kolom", lambda: con_d.sql(q_satu).fetchall())
t_banyak = ukur("latihan: SUM beberapa kolom", lambda: con_d.sql(q_banyak).fetchall())

print("hasil satu kolom:", t_satu)
print("hasil beberapa kolom:", t_banyak)

```

Output tersimpan

```

[latihan: SUM satu kolom] 0.0855 s
[latihan: SUM beberapa kolom] 0.1695 s
hasil satu kolom: [(81621442.45996779,)]
hasil beberapa kolom: [(81621442.45996639, 10231889.610000085, 497452755, 491330400)]

```

Pembahasan

SUM satu kolom memerlukan 0,0855 detik dan SUM empat kolom 0,1695 detik, sekitar 1,98 kali. Pembacaan kolom tambahan sesuai konsep column pruning, tetapi waktu overhead dan cache belum dipisahkan. Hasil total_bayar berbeda sekitar 0,0000014 pada digit belakang; perbedaan urutan penjumlahan float adalah kemungkinan penjelasan, bukan hasil pemeriksaan ulang dataset. Penjumlahan ID zona pada latihan hanya memperbanyak kolom baca, tidak memiliki makna pendapatan.

4.12.4 Out Degree Graph dan SQL

Cell P26. Out Degree Graph dan SQL (cell notebook 54)

```

top_out_graph = sorted(
    G.out_degree(), key=lambda x: (-x[1], x[0])
)[:5]

top_out_sql = con_d.sql("""
    SELECT asal, COUNT(DISTINCT tujuan) AS jumlah_tujuan
    FROM arus_df
    GROUP BY asal
    ORDER BY jumlah_tujuan DESC, asal
    LIMIT 5
""").fetchall()

print("Graph:", top_out_graph)
print("SQL :", top_out_sql)
print("hasil sama?", top_out_graph == top_out_sql)

```

Output tersimpan

```
Graph: [(132, 260), (138, 254), (264, 241), (230, 234), (140, 233)]  
SQL : [(132, 260), (138, 254), (264, 241), (230, 234), (140, 233)]  
hasil sama? True
```

Pembahasan

Lima zona teratas adalah 132, 138, 264, 230, dan 140, dengan 260, 254, 241, 234, dan 233 tujuan berbeda. Output graph dan SQL sama. Out-degree tanpa bobot menghitung edge keluar, termasuk self-loop jika tersedia, bukan jumlah perjalanan.

4.12.5 Ukuran Kotak Spasial

Cell P27. Membandingkan Ukuran Kotak (cell notebook 57)

```
def uji_sisi_spasial(setengah_sisi):  
    kotak_uji = [  
        box(x - setengah_sisi, y - setengah_sisi,  
            x + setengah_sisi, y + setengah_sisi)  
        for x, y in zip(cx, cy)  
    ]  
  
    t0 = time.perf_counter()  
    hasil_numpy = []  
    for x, y in zip(cx, cy):  
        mask = (  
            (bujur >= x - setengah_sisi)  
            & (bujur <= x + setengah_sisi)  
            & (lintang >= y - setengah_sisi)  
            & (lintang <= y + setengah_sisi)  
        )  
        hasil_numpy.append(int(mask.sum()))  
    waktu_numpy = time.perf_counter() - t0  
  
    t0 = time.perf_counter()  
    hasil_indeks = [  
        len(pohon.query(k, predicate="contains")) for k in kotak_uji  
    ]  
    waktu_indeks = time.perf_counter() - t0  
  
    return {  
        "panjang_sisi_derajat": setengah_sisi * 2,  
        "waktu_numpy": waktu_numpy,  
        "waktu_stree": waktu_indeks,  
        "rasio_numpy_per_stree": waktu_numpy / waktu_indeks,  
        "hasil_sama": hasil_numpy == hasil_indeks,  
    }  
  
hasil_sisi = pd.DataFrame([  
    uji_sisi_spasial(0.01),  
    uji_sisi_spasial(0.0005),  
)  
]  
display(hasil_sisi)
```

Output tersimpan

	panjang_sisi_derajat	waktu_numpy	waktu_strtree	rasio_numpy_per_strtree \
0	0.020	0.524885	0.774640	0.677586
1	0.001	0.526047	0.019157	27.459112

	hasil_sama
0	True
1	True

Pembahasan

Untuk sisi 0,02 derajat, NumPy 0,524885 detik dan STRtree 0,774640 detik sehingga rasio NumPy/STRtree 0,677586: STRtree lebih lambat. Untuk sisi 0,001 derajat, NumPy 0,526047 detik dan STRtree 0,019157 detik sehingga STRtree sekitar 27,46 kali lebih cepat. Hasilnya sama pada kedua ukuran. Kotak kecil menyisihkan lebih banyak kandidat; biaya membangun indeks tidak diulang dalam pengujian ini.

4.12.6 Time Travel dan Restore

Cell P28. Restore ke Versi 1 (cell notebook 59)

```
dt = DeltaTable(PATH_DELTA)
print("versi sebelum restore:", dt.version(), "| baris:", len(dt.to_pandas()))

dt.restore(1)
dt = DeltaTable(PATH_DELTA)

print("versi setelah restore:", dt.version(), "| baris:", len(dt.to_pandas()))
print([h.get("operation") for h in dt.history()][:3])
```

Output tersimpan

```
versi sebelum restore: 23 | baris: 160000
versi setelah restore: 24 | baris: 150000
['RESTORE', 'OPTIMIZE', 'WRITE']
```

Pembahasan

Restore dari versi 23 dengan 160.000 baris ke keadaan versi 1 menghasilkan versi baru 24 dengan 150.000 baris. Riwayat teratas RESTORE, OPTIMIZE, WRITE. Time travel hanya membaca versi tertentu, sedangkan restore mengubah keadaan terbaru. Restore(1) juga mengembalikan keadaan sebelum koreksi Bronx, sehingga tujuan versi harus dipilih sesuai koreksi yang hendak dibatalkan.

4.13 Penyimpanan Arsip Delta

Cell P29. Membuat Arsip Delta (cell notebook 62)

```
PATH_ZIP_DELTA = f"{DIR_SIMPAN}/trips_delta"
hasil_zip = shutil.make_archive(PATH_ZIP_DELTA, "zip", root_dir=PATH_DELTA)
print("Arsip Delta tersimpan:", hasil_zip)
```

Output tersimpan

Arsip Delta tersimpan: /content/drive/MyDrive/BigData/Praktikum4/trips_delta.zip

Output mencatat arsip trips_delta.zip pada folder Drive Praktikum4. Arsip dibuat setelah latihan restore, sehingga keadaan terbaru yang diarsipkan mengikuti versi 24. Berkas ZIP fisik tidak disertakan pada lampiran sumber laporan ini; isinya tidak diperiksa ulang.

BAB V

TUGAS PRAKTIKUM DAN PERTANYAAN EVALUASI

5.1 Persiapan Notebook Tugas

Notebook tugas berdiri terpisah dari notebook praktikum. Nomor T menunjukkan cell kode tugas pada laporan. Output tugas berasal dari notebook tersebut dan tidak digabung sebagai pengukuran ulang K-2 hingga K-9.

Cell T1. Instalasi Library Tugas (cell notebook 3)

```
!pip install -q duckdb networkx tinydb "deltalake>=1.0"
```

Output tersimpan

Output tidak tersimpan pada notebook untuk cell ini.

Cell T2. Lingkungan dan Sumber Data Tugas (cell notebook 4)

```
from google.colab import drive
drive.mount("/content/drive")

import os
import json
import time
import shutil
import sqlite3
import zipfile

import pandas as pd
from IPython.display import display, Markdown

DIR_P3 = "/content/drive/MyDrive/BigData/Praktikum3"
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum4/TugasMandiri"
TRIPS_BERSIH_P4 = f"{DIR_SIMPAN}/trips_bersih"
TRIPS_BERSIH_P3 = f"{DIR_P3}/trips_bersih"
ZIP_TRIPS = f"{DIR_P3}/trips_bersih.zip"

os.makedirs(DIR_SIMPAN, exist_ok=True)

if os.path.isdir(TRIPS_BERSIH_P4):
    TRIPS_BERSIH = TRIPS_BERSIH_P4
else:
    TRIPS_BERSIH = TRIPS_BERSIH_P3

if not os.path.exists(TRIPS_BERSIH) and os.path.exists(ZIP_TRIPS):
    HASIL_EKSTRAK = "/content/trips_bersih_p04"
    os.makedirs(HASIL_EKSTRAK, exist_ok=True)
    with zipfile.ZipFile(ZIP_TRIPS, "r") as arsip:
        arsip.extractall(HASIL_EKSTRAK)
    kandidat = [os.path.join(HASIL_EKSTRAK, "trips_bersih"), HASIL_EKSTRAK]
    TRIPS_BERSIH = next(
        (p for p in kandidat if any(f.endswith(".parquet") for _, _, fs in os.walk(p) for f in fs)),
```

```

    TRIPS_BERSIH,
)

print("Folder Praktikum 4:", DIR_SIMPAN)
print("Dataset:", TRIPS_BERSIH)

catatan = []

def ukur(label, fungsi):
    t0 = time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    catatan.append({"langkah": label, "detik": round(detik, 4)})
    print(f"[{label}] {detik:.4f} s")
    return hasil

```

Output tersimpan

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Folder Praktikum 4: /content/drive/MyDrive/BigData/Praktikum4/TugasMandiri
Dataset: /content/trips_bersih_p04

5.2 Rekap Benchmark R-1

Rekap membaca CSV hasil notebook praktikum dan menghasilkan enam baris kebutuhan beserta interpretasinya. Angka yang tidak tersimpan pada CSV, seperti jumlah dokumen, jumlah zona terjangkau, dan versi Delta, diambil dari output notebook utama.

Cell T3. Membaca Benchmark Praktikum (cell notebook 6)

```

PATH_BENCHMARK = "/content/drive/MyDrive/BigData/Praktikum4/benchmark_nosql.csv"
"
assert os.path.exists(PATH_BENCHMARK), "benchmark_nosql.csv belum ada di folder Praktikum4."

bench = pd.read_csv(PATH_BENCHMARK)
waktu = bench.set_index("langkah")["detik"]
display(bench)

```

Output tersimpan

	langkah	detik
0	baca trips_bersih	2.7022
1	relasional: muat tabel	2.5168
2	key-value: bangun dict	9.6995
3	cari 200 perjalanan - SQL tanpa indeks	4.3215
4	cari 200 perjalanan - SQL dengan indeks	0.0037
5	cari 200 perjalanan - key-value	0.0012
6	hitung per wilayah - SQL GROUP BY	0.1951
7	hitung per wilayah - key-value (buka semua nilai)	1.4229
8	DuckDB - hitung per wilayah (langsung dari Par...	0.1139

9	SQLite - muat seluruh baris ke tabel	9.1240
10	SQLite - hitung per wilayah	1.7269
11	DuckDB - hanya wilayah Staten Island	0.0176
12	hitung arus antar zona	0.2170
13	2 langkah - graph (NetworkX)	0.0068
14	2 langkah - SQL self-join (DuckDB)	0.0081
15	cara 1 - loop Python, 5 kotak	13.9702
16	cara 2 - NumPy, 1000 kotak	0.5210
17	bangun indeks STRtree	0.0788
18	cara 3 - STRtree, 1000 kotak	0.2139

Cell T4. Menyusun Enam Baris Rekap (cell notebook 7)

```
def idn(x, desimal=4):
    teks = f"{x:,.{desimal}f}"
    return teks.replace(",", "X").replace(".", ",").replace("X", ".")

def kali(a, b, desimal=1):
    return idn(waktu[a] / waktu[b], desimal)

K_TANPA = "cari 200 perjalanan - SQL tanpa indeks"
K_INDEKS = "cari 200 perjalanan - SQL dengan indeks"
K_KV = "cari 200 perjalanan - key-value"
K_HITUNG_SQL = "hitung per wilayah - SQL GROUP BY"
K_HITUNG_KV = "hitung per wilayah - key-value (buka semua nilai)"
K_DUCK = "DuckDB - hitung per wilayah (langsung dari Parquet)"
K_SQLITE = "SQLite - hitung per wilayah"
K_GRAF = "2 langkah - graph (NetworkX)"
K_SJOIN = "2 langkah - SQL self-join (DuckDB)"
K_LOOP = "cara 1 - loop Python, 5 kotak"
K_NUMPY = "cara 2 - NumPy, 1000 kotak"
K_BANGUN = "bangun indeks STRtree"
K_STR = "cara 3 - STRtree, 1000 kotak"

total_str = waktu[K_BANGUN] + waktu[K_STR]
loop_1000 = waktu[K_LOOP] / 5 * 1000

baris = [
    {
        "Kebutuhan": "Ambil satu data lewat id",
        "Cara yang unggul": "Key-value; tabel dengan indeks",
        "Cara yang lemah": "Tabel tanpa indeks",
        "Angka saya": (
            f"200 pencarian: key-value {idn(waktu[K_KV])} s; tabel berindeks {i"
            dn(waktu[K_INDEKS])} s; "
            f"tabel tanpa indeks {idn(waktu[K_TANPA])} s"
        ),
        "Interpretasi": (
            f"Dengan indeks, tabel SQL hanya {kali(K_INDEKS, K_KV)} kali lebih"
            lambat dari key-value, "
            f"dan {kali(K_TANPA, K_INDEKS, 0)} kali lebih cepat daripada yang t"
            anpa indeks. "
        )
    }
]
```

```

        f"Jadi yang menentukan adalah ada tidaknya indeks, bukan jenis mode
Inya."
    ),
},
{
    "Kebutuhan": "Menghitung atau merangkum banyak baris",
    "Cara yang unggul": "Per kolom (DuckDB pada Parquet)",
    "Cara yang lemah": "Key-value (buka semua nilai)",
    "Angka saya": (
        f"300.000 baris: SQL GROUP BY {idn(waktu[K_HITUNG_SQL])} s, key-val
ue {idn(waktu[K_HITUNG_KV])} s. "
        f"2.986.910 baris: DuckDB {idn(waktu[K_DUCK])} s, SQLite {idn(waktu
[K_SQLITE])} s"
    ),
    "Interpretasi": (
        f"Key-value {kali(K_HITUNG_KV, K_HITUNG_SQL)} kali lebih lambat dar
i SQL karena tiap nilai JSON "
        f"harus dibuka dulu. DuckDB {kali(K_SQLITE, K_DUCK)} kali lebih cep
at dari SQLite "
        f"karena hanya membaca kolom yang dipakai dan memproses banyak bari
s sekaligus."
    ),
},
{
    "Kebutuhan": "Data dengan field opsional atau berubah-ubah",
    "Cara yang unggul": "Document",
    "Cara yang lemah": "Tabel (kolom NULL, migrasi)",
    "Angka saya": "4.094 dari 20.000 dokumen tanpa field tip; 1 dokumen mem
bawa field cuaca (output K-3)",
    "Interpretasi": (
        "Document menerima field baru atau field yang tidak selalu ada tanp
a migrasi. Tapi "
        "salah ketik nama field juga diterima begitu saja tanpa peringatan,
jadi validasinya pindah ke aplikasi."
    ),
},
{
    "Kebutuhan": "Keterhubungan dengan jumlah langkah bebas",
    "Cara yang unggul": "Graph",
    "Cara yang lemah": "SQL (satu JOIN per langkah)",
    "Angka saya": (
        f"2 langkah: graph {idn(waktu[K_GRAF])} s, SQL {idn(waktu[K_SJOIN])
} s. "
        f"Zona terjangkau untuk k = 1 sampai 4: 74, 214, 214, 214 (output K
-5)"
    ),
    "Interpretasi": (
        f"Kecepatannya hampir sama (SQL {kali(K_SJOIN, K_GRAF, 2)} kali gra
ph). Kelebihan graph ada di "
        f"cara bertanya: ganti k cukup ubah satu angka, sedangkan SQL harus
menambah satu JOIN tiap langkah."
    ),
},

```

```

    },
    {
        "Kebutuhan": "Mencari lokasi dalam wilayah",
        "Cara yang unggul": "Spasial dengan indeks",
        "Cara yang lemah": "Memeriksa semua titik",
        "Angka saya": (
            f"1.000 kotak: STRtree {idn(total_str)} s (termasuk bangun indeks);
NumPy {idn(waktu[K_NUMPY])} s; "
            f"loop Python perkiraan {idn(loop_1000, 0)} s"
        ),
        "Interpretasi": (
            f"STRtree {idn(waktu[K_NUMPY] / total_str, 1)} kali lebih cepat dar
i NumPy dan ribuan kali dari loop. "
            f"Tapi pada kotak 0,02 derajat ia malah 0,68 kali (lebih lambat), d
an pada kotak 0,001 derajat 27,5 kali "
            f"(Latihan 5). Jadi klaim percepatan harus menyebut pembandingnya."

        ),
    },
    {
        "Kebutuhan": "Perubahan yang harus aman dan bisa dibatalkan",
        "Cara yang unggul": "Tabel Delta",
        "Cara yang lemah": "Folder Parquet biasa",
        "Angka saya": "Update 20 baris: selisih total_bayar 20,0; kolom asing d
itolak, versi tetap 2 (output K-7)",
        "Interpretasi": (
            "Versi lama tetap bisa dibaca setelah update, dan penulisan yang ga
gal tidak meninggalkan data setengah jadi. "
            "Dua hal ini tidak dimiliki folder Parquet biasa."
        ),
    },
},
]

rekap = pd.DataFrame(baris)
print("jumlah baris rekap:", len(rekap))

```

Output tersimpan

jumlah baris rekap: 6

Cell T5. Menampilkan dan Menyimpan Rekap (cell notebook 8)

```

kolom = ["Kebutuhan", "Cara yang unggul", "Cara yang lemah", "Angka saya", "Int
erpretasi"]
rekap = rekap[kolom]

tabel_md = ["| " + " | ".join(kolom) + " |", "|" + "---|" * len(kolom)]
for _, r in rekap.iterrows():
    tabel_md.append("| " + " | ".join(str(r[k]).replace("|", "/") for k in kolo
m) + " |")
display(Markdown("\n".join(tabel_md)))

```

```

rekap.to_csv(f"{DIR_SIMPAN}/rekap_k9.csv", index=False)
print("Tersimpan:", f"{DIR_SIMPAN}/rekap_k9.csv")

```

Output tersimpan

Kebutuhan	Cara yang unggul	Cara yang lemah	Angka saya	Interpretasi
Ambil satu data lewat id	Key-value; tabel dengan indeks	Tabel tanpa indeks	200 pencarian: key-value 0,0012 s; tabel berindeks 0,0037 s; tabel tanpa indeks 4,3215 s	Dengan indeks, tabel SQL hanya 3,1 kali lebih lambat dari key-value, dan 1.168 kali lebih cepat daripada yang tanpa indeks. Jadi yang menentukan adalah ada tidaknya indeks, bukan jenis modelnya.
Menghitung atau merangkum banyak baris	Per kolom (DuckDB pada Parquet)	Key-value (buka semua nilai)	300.000 baris: SQL GROUP BY 0,1951 s, key-value 1,4229 s. 2.986.910 baris: DuckDB 0,1139 s, SQLite 1,7269 s	Key-value 7,3 kali lebih lambat dari SQL karena tiap nilai JSON harus dibuka dulu. DuckDB 15,2 kali lebih cepat dari SQLite karena hanya membaca kolom yang dipakai dan memproses banyak baris sekaligus.
Data dengan field opsional atau berubah-ubah	Document	Tabel (kolom NULL, migrasi)	4.094 dari 20.000 dokumen tanpa field tip; 1 dokumen membawa field cuaca (output K-3)	Document menerima field baru atau field yang tidak selalu ada tanpa migrasi. Tapi salah ketik nama field juga diterima begitu saja tanpa peringatan, jadi validasinya pindah ke aplikasi.
Keterhubungan dengan jumlah langkah bebas	Graph	SQL (satu JOIN per langkah)	2 langkah: graph 0,0068 s, SQL 0,0081 s. Zona terjangkau untuk k = 1 sampai 4: 74,	Kecepatannya hampir sama (SQL 1,19 kali graph). Kelebihan graph ada di cara bertanya: ganti k

Kebutuhan	Cara yang unggul	Cara yang lemah	Angka saya	Interpretasi
			214, 214, 214 (output K-5)	cukup ubah satu angka, sedangkan SQL harus menambah satu JOIN tiap langkah.
Mencari lokasi dalam wilayah	Spasial dengan indeks	Memeriksa semua titik	1.000 kotak: STRtree 0,2927 s (termasuk bangun indeks); NumPy 0,5210 s; loop Python perkiraan 2.794 s	STRtree 1,8 kali lebih cepat dari NumPy dan ribuan kali dari loop. Tapi pada kotak 0,02 derajat ia malah 0,68 kali (lebih lambat), dan pada kotak 0,001 derajat 27,5 kali (Latihan 5). Jadi klaim percepatan harus menyebut pembandingnya.
Perubahan yang harus aman dan bisa dibatalkan	Tabel Delta	Folder Parquet biasa	Update 20 baris: selisih total_bayar 20,0; kolom asing ditolak, versi tetap 2 (output K-7)	Versi lama tetap bisa dibaca setelah update, dan penulisan yang gagal tidak meninggalkan data setengah jadi. Dua hal ini tidak dimiliki folder Parquet biasa.

Tersimpan: /content/drive/MyDrive/BigData/Praktikum4/TugasMandiri/rekap_k9.csv

Kebutuhan	Pilihan dan hasil	Interpretasi
Pencarian ID	Key-value 0,0012 s; SQL berindeks 0,0037 s	Indeks menghindari pemindaian; selisih keduanya kecil.
Agregasi	300.000 baris: SQL 0,1951 s vs key-value 1,4229 s. Seluruh data: DuckDB 0,1139 s vs SQLite 1,7269 s.	Bandingkan dalam pasangan dengan cakupan sama.
Field opsional	Document: 4.094 dokumen awal tidak memiliki tip	Fleksibel; perlu validasi aplikasi.
Keterhubungan	Graph 0,0068 s vs SQL 0,0081 s	Kemudahan formulasi jumlah langkah menjadi pertimbangan.

Kebutuhan	Pilihan dan hasil	Interpretasi
Wilayah spasial	NumPy 0,5210 s vs STRtree total 0,2927 s	Keuntungan bergantung ukuran wilayah dan pembandingan.
Perubahan dan audit	Delta: update 20 baris; selisih 20,0; penolakan skema versi tetap 2	Log memungkinkan pembacaan versi lama dan komit yang terkontrol.

5.3 Data Zona dalam Tiga Model R-2

Data zona dibentuk dari zona penjemputan dan ringkasan arus seluruh perjalanan. Terdapat 254 zona penjemputan dan 21.583 pasangan arus. Graph memiliki 262 node karena mencakup zona asal maupun tujuan, sedangkan tabel referensi zona dan dokumen hanya mencakup zona asal. Perbedaan jumlah objek tersebut bukan bukti data hilang.

Cell T6. Membentuk Data Zona dan Arus (cell notebook 10)

```
import glob
import duckdb
import networkx as nx
from tinydb import TinyDB, Query
from tinydb.storages import MemoryStorage

GLOB = f"{TRIPS_BERSIH}/**/*.parquet"
if not glob.glob(GLOB):
    GLOB = f"{TRIPS_BERSIH}/**/*.parquet"

con_d = duckdb.connect()
con_d.sql(f"""
CREATE OR REPLACE VIEW perjalanan AS
SELECT tpep_pickup_datetime AS waktu_jemput,
       PULocationID        AS zona_jemput,
       DOLocationID        AS zona_tujuan,
       CASE WHEN borough_naik IS NULL
            OR borough_naik = '__HIVE_DEFAULT_PARTITION__'
            THEN 'Unknown' ELSE borough_naik END AS wilayah_jemput,
       trip_distance        AS jarak_mil,
       total_amount         AS total_bayar,
       nama_pembayaran      AS metode_bayar
FROM read_parquet('{GLOB}', hive_partitioning=true)
""")

arus = ukur("hitung arus antar zona", lambda: con_d.sql("""
SELECT zona_jemput AS asal, zona_tujuan AS tujuan, COUNT(*) AS jumlah
FROM perjalanan
GROUP BY 1, 2
""").df())

zona_ref = con_d.sql("""
SELECT zona_jemput AS zona, MIN(wilayah_jemput) AS wilayah
FROM perjalanan
""")
```

```

GROUP BY 1
""").df()

arus["asal"] = arus["asal"].astype(int)
arus["tujuan"] = arus["tujuan"].astype(int)
arus["jumlah"] = arus["jumlah"].astype(int)
zona_ref["zona"] = zona_ref["zona"].astype(int)
wilayah_zona = dict(zip(zona_ref["zona"], zona_ref["wilayah"]))

populer = (
    arus.groupby("tujuan", as_index=False)["jumlah"].sum()
    .sort_values(["jumlah", "tujuan"], ascending=[False, True])
)
zona_awal = int(populer.iloc[0]["tujuan"])
MIN = 50

print("zona:", len(zona_ref), "| arus:", len(arus), "| zona awal (tujuan terpopuler):", zona_awal)

```

Output tersimpan

[hitung arus antar zona] 0.2041 s
zona: 254 | arus: 21583 | zona awal (tujuan terpopuler): 236

Cell T7. Menyimpan Zona dalam Tiga Model (cell notebook 12)

```

# 1. Tabel
con_s = sqlite3.connect(":memory:")
zona_ref.to_sql("zona", con_s, index=False)
arus.to_sql("arus", con_s, index=False)
con_s.execute("CREATE INDEX idx_arus_asal ON arus(asal)")

# 2. Document
db = TinyDB(storage=MemoryStorage)
tabel_zona = db.table("zona")
keluar = {}
urut = arus.sort_values(["asal", "jumlah", "tujuan"], ascending=[True, False, True])
for r in urut.itertuples(index=False):
    keluar.setdefault(int(r.asal), []).append({"tujuan": int(r.tujuan), "jumlah": int(r.jumlah)})

tabel_zona.insert_multiple([
    {
        "zona": z,
        "wilayah": wilayah_zona.get(z, "Unknown"),
        "total_keluar": sum(t["jumlah"] for t in daftar),
        "keluar": daftar,
    }
    for z, daftar in keluar.items()
])

# 3. Graph

```

```

G = nx.DiGraph()
for r in arus.itertuples(index=False):
    G.add_edge(int(r.asal), int(r.tujuan), jumlah=int(r.jumlah))
nx.set_node_attributes(G, wilayah_zona, "wilayah")

print("tabel zona:", con_s.execute("SELECT COUNT(*) FROM zona").fetchone()[0],
      "| tabel arus:", con_s.execute("SELECT COUNT(*) FROM arus").fetchone()[0]
    )
print("dokumen zona:", len(tabel_zona))
print("node:", G.number_of_nodes(), "| edge:", G.number_of_edges())

```

Output tersimpan

```

tabel zona: 254 | tabel arus: 21583
dokumen zona: 254
node: 262 | edge: 21583

```

Cell T8. Agregasi Wilayah pada Tabel (cell notebook 14)

```

def keluar_per_wilayah_tabel():
    return dict(con_s.execute("""
        SELECT z.wilayah, SUM(a.jumlah)
        FROM arus a
        JOIN zona z ON z.zona = a.asal
        GROUP BY z.wilayah
        """).fetchall())

hasil_tabel_mudah = ukur("tabel (mudah): perjalanan keluar per wilayah", keluar_per_wilayah_tabel)
print(sorted(hasil_tabel_mudah.items(), key=lambda x: (-x[1], x[0])))

```

Output tersimpan

```

[tabel (mudah): perjalanan keluar per wilayah] 0.0258 s
[('Manhattan', 2659609), ('Queens', 270315), ('Unknown', 37949), ('Brooklyn', 15724), ('Bronx', 3074), ('Staten Island', 211), ('EWR', 28)]

```

Cell T9. Keterjangkauan Tiga Langkah dengan SQL (cell notebook 16)

```

def tiga_langkah_tabel():
    baris = con_s.execute("""
        WITH a AS (SELECT asal, tujuan FROM arus WHERE jumlah >= :m)
        SELECT tujuan FROM a WHERE asal = :s
        UNION
        SELECT a2.tujuan
        FROM a a1 JOIN a a2 ON a1.tujuan = a2.asal
        WHERE a1.asal = :s
        UNION
        SELECT a3.tujuan
        FROM a a1 JOIN a a2 ON a1.tujuan = a2.asal
             JOIN a a3 ON a2.tujuan = a3.asal
        WHERE a1.asal = :s
    """)

```

```

"""", {"s": zona_awal, "m": MIN}).fetchall()
return {r[0] for r in baris} - {zona_awal}

hasil_tabel_canggung = ukur("tabel (canggung): terjangkau <= 3 langkah", tiga_l
angkah_tabel)
print("zona terjangkau:", len(hasil_tabel_canggung))

```

Output tersimpan

```

[tabel (canggung): terjangkau <= 3 langkah] 0.2718 s
zona terjangkau: 214

```

Cell T10. Profil Zona pada Document (cell notebook 18)

```

Q = Query()

def profil_zona(z):
    return tabel_zona.get(Q.zona == z)

dok = ukur("document (mudah): profil satu zona", lambda: profil_zona(zona_awal)
)
print("zona:", dok["zona"], "| wilayah:", dok["wilayah"],
      "| tujuan berbeda:", len(dok["keluar"]), "| total keluar:", dok["total_ke
luar"])
print("tiga tujuan teratas:", dok["keluar"][:3])

```

Output tersimpan

```

[document (mudah): profil satu zona] 0.0002 s
zona: 236 | wilayah: Manhattan | tujuan berbeda: 203 | total keluar: 136276
tiga tujuan teratas: [{ 'tujuan': 237, 'jumlah': 18810}, { 'tujuan': 236, 'jumlah
': 14110}, { 'tujuan': 161, 'jumlah': 7497}]

```

Cell T11. Pengirim Terbesar pada Document dan SQL (cell notebook 20)

```

def lima_pengirim_terbesar_doc(z):
    hasil = []
    for d in tabel_zona.all():
        for t in d["keluar"]:
            if t["tujuan"] == z:
                hasil.append((d["zona"], t["jumlah"]))
    return sorted(hasil, key=lambda x: (-x[1], x[0]))[:5]

def lima_pengirim_terbesar_sql(z):
    return con_s.execute("""
        SELECT asal, jumlah FROM arus
        WHERE tujuan = ?
        ORDER BY jumlah DESC, asal
        LIMIT 5
        """, (z,)).fetchall()

h_doc = ukur("document (canggung): pengirim terbesar ke satu zona", lambda: lim

```

```

a_pengirim_terbesar_doc(zona_awal))
h_sql = lima_pengirim_terbesar_sql(zona_awal)
print("document:", h_doc)
print("SQL    :", [tuple(r) for r in h_sql])
print("hasil sama?", h_doc == [tuple(r) for r in h_sql])

```

Output tersimpan

```

[document (canggung): pengirim terbesar ke satu zona] 0.0051 s
document: [(237, 22065), (236, 14110), (161, 8524), (141, 8232), (263, 6371)]
SQL    : [(237, 22065), (236, 14110), (161, 8524), (141, 8232), (263, 6371)]
hasil sama? True

```

Cell T12. Keterjangkauan Tiga Langkah pada Graph (cell notebook 22)

```

H = nx.DiGraph([(a, b) for a, b, d in G.edges(data=True) if d["jumlah"] >= MIN]
)

def tiga_langkah_graph():
    if zona_awal not in H:
        return set()
    return set(nx.single_source_shortest_path_length(H, zona_awal, cutoff=3)) -
    {zona_awal}

hasil_graph_mudah = ukur("graph (mudah): terjangkau <= 3 langkah", tiga_langkah
_graph)
print("zona terjangkau:", len(hasil_graph_mudah))
print("sama dengan jawaban tabel?", hasil_graph_mudah == hasil_tabel_canggung)

```

Output tersimpan

```

[graph (mudah): terjangkau <= 3 langkah] 0.0005 s
zona terjangkau: 214
sama dengan jawaban tabel? True

```

Cell T13. Agregasi Wilayah pada Graph (cell notebook 24)

```

def keluar_per_wilayah_graph():
    total = {}
    for asal, tujuan, d in G.edges(data=True):
        w = G.nodes[asal]["wilayah"]
        total[w] = total.get(w, 0) + d["jumlah"]
    return total

hasil_graph_canggung = ukur("graph (canggung): perjalanan keluar per wilayah",
keluar_per_wilayah_graph)
print(sorted(hasil_graph_canggung.items(), key=lambda x: (-x[1], x[0])))
print("sama dengan jawaban tabel?", hasil_graph_canggung == hasil_tabel_mudah)

```

Output tersimpan

```

[graph (canggung): perjalanan keluar per wilayah] 0.0170 s

```

[('Manhattan', 2659609), ('Queens', 270315), ('Unknown', 37949), ('Brooklyn', 15724), ('Bronx', 3074), ('Staten Island', 211), ('EWR', 28)]
sama dengan jawaban tabel? True

Model	Pertanyaan mudah	Pertanyaan canggung
Tabel	Jumlah perjalanan keluar per wilayah: 0,0258 s	Keterjangkauan paling banyak 3 langkah: 0,2718 s
Document	Profil zona 236: 0,0002 s	Lima pengirim terbesar ke zona 236: 0,0051 s
Graph	Keterjangkauan paling banyak 3 langkah: 0,0005 s	Perjalanan keluar per wilayah: 0,0170 s

Graph dan SQL memberikan 214 zona terjangkau untuk paling banyak tiga langkah. Formulasi graph lebih singkat dan pada tugas ini waktunya lebih rendah, tetapi selisih bukan akibat model saja: engine, kode, dan indeks juga berbeda. Profil dokumen zona 236 memuat 203 tujuan dan 136.276 perjalanan keluar. Lima pengirim terbesar pada dokumen sama dengan SQL. Agregasi wilayah graph juga sama dengan tabel.

Referensi wilayah menggunakan MIN(wilayah_jemput) untuk satu nilai per zona; notebook tidak mencetak pemeriksaan bahwa setiap zona hanya memiliki satu label wilayah. Dalam pemakaian lanjutan, pemetaan sebaiknya divalidasi atau menggunakan referensi zona resmi. Node yang hanya menjadi tujuan tidak diberi atribut wilayah melalui pemetaan asal, tetapi query keluar per wilayah hanya membaca node asal sehingga output yang tersimpan tetap berhasil.

5.4 Simulasi Delta Lake Dua Hari R-3

Cell T14. Sampel Delta Dua Hari (cell notebook 27)

```
from deltalake import DeltaTable, write_deltalake, CommitProperties

data_delta = con_d.sql("""
    SELECT waktu_jemput, zona_jemput, zona_tujuan, wilayah_jemput,
           jarak_mil, total_bayar, metode_bayar
    FROM perjalanan
    USING SAMPLE reservoir(100000 ROWS) REPEATABLE (42)
""").df().reset_index(drop=True)
print("baris sampel:", f"{len(data_delta):,}")

PATH_2HARI = os.path.join(os.getcwd(), "trips_delta_2hari")
if os.path.exists(PATH_2HARI):
    shutil.rmtree(PATH_2HARI)

def label(hari, ket):
    return CommitProperties(custom_metadata={"hari": str(hari), "keterangan": k
et})
```

Output tersimpan

baris sampel: 100,000

Cell T15. Menulis Batch Hari Pertama (cell notebook 29)

```
pagi = data_delta.iloc[:40_000]
sore = data_delta.iloc[40_000:60_000]

write_deltalake(PATH_2HARI, pagi, mode="overwrite", commit_properties=label(1,
"batch pagi"))
write_deltalake(PATH_2HARI, sore, mode="append", commit_properties=label(1, "ba
tch sore"))

dt = DeltaTable(PATH_2HARI)
VERSI_AKHIR_HARI1 = dt.version()
akhir_hari1 = dt.to_pandas()
BARIS_HARI1 = len(akhir_hari1)
TOTAL_HARI1 = float(akhir_hari1["total_bayar"].sum())

print("versi akhir hari 1:", VERSI_AKHIR_HARI1)
print("baris akhir hari 1:", f"{BARIS_HARI1:,}")
print("total_bayar akhir hari 1:", round(TOTAL_HARI1, 2))
```

Output tersimpan

versi akhir hari 1: 1
baris akhir hari 1: 60,000
total_bayar akhir hari 1: 1559049.94

Cell T16. Penambahan dan Koreksi Hari Kedua (cell notebook 31)

```
hari2 = data_delta.iloc[60_000:100_000]
write_deltalake(PATH_2HARI, hari2, mode="append", commit_properties=label(2, "t
ambah data hari kedua"))

tersedia = data_delta.iloc[:100_000]
n_koreksi = int(((tersedia["wilayah_jemput"] == "Queens") & (tersedia["metode_b
ayar"] == "Tunai")).sum())
assert n_koreksi > 0, "Tidak ada baris Queens + Tunai pada sampel. Ganti predik
at koreksi."
print("baris yang akan dikoreksi:", n_koreksi)

dt = DeltaTable(PATH_2HARI)
dt.update(
    predicate="wilayah_jemput = 'Queens' AND metode_bayar = 'Tunai'",
    updates={"total_bayar": "total_bayar + 1.0"},
    commit_properties=label(2, "koreksi tarif Tunai Queens"),
)

terbaru = DeltaTable(PATH_2HARI).to_pandas()
print("versi terbaru:", DeltaTable(PATH_2HARI).version(), "| baris:", f"{len(te
rbaru):,}")
```

Output tersimpan

baris yang akan dikoreksi: 704
versi terbaru: 3 | baris: 100,000

Cell T17. Time Travel ke Akhir Hari Pertama (cell notebook 33)

```
h1 = DeltaTable(PATH_2HARI, version=VERSI_AKHIR_HARI1).to_pandas()
total_h1_baca = float(h1["total_bayar"].sum())

print("time travel ke versi", VERSI_AKHIR_HARI1)
print("baris hasil baca      :", f"{len(h1):,}", "| catatan akhir hari 1:", f"{
BARIS_HARI1:,}")
print("total_bayar hasil baca:", round(total_h1_baca, 2), "| catatan akhir hari
1:", round(TOTAL_HARI1, 2))
print("sama dengan keadaan akhir hari 1?",
      len(h1) == BARIS_HARI1 and abs(total_h1_baca - TOTAL_HARI1) < 0.01)

selisih = float(terbaru["total_bayar"].sum()) - total_h1_baca
harapan = float(hari2["total_bayar"].sum()) + n_koreksi * 1.0
print("selisih total_bayar (terbaru - akhir hari 1):", round(selisih, 2))
print("total hari 2 + koreksi          :", round(harapan, 2))
print("selisih sesuai?", abs(selisih - harapan) < 0.01)
```

Output tersimpan

time travel ke versi 1
baris hasil baca : 60,000 | catatan akhir hari 1: 60,000
total_bayar hasil baca: 1559049.94 | catatan akhir hari 1: 1559049.94
sama dengan keadaan akhir hari 1? True
selisih total_bayar (terbaru - akhir hari 1): 1038019.97
total hari 2 + koreksi : 1038019.97
selisih sesuai? True

Cell T18. Riwayat Komit Dua Hari (cell notebook 35)

```
for h in DeltaTable(PATH_2HARI).history():
    print(h.get("version"), h.get("operation"), "| hari:", h.get("hari"), "|",
          h.get("keterangan"))
```

Output tersimpan

3 UPDATE | hari: 2 | koreksi tarif Tunai Queens
2 WRITE | hari: 2 | tambah data hari kedua
1 WRITE | hari: 1 | batch sore
0 WRITE | hari: 1 | batch pagi

Cell T19. Mencetak Isi Log Transaksi (cell notebook 36)

```
PATH_LOG = f"{PATH_2HARI}/_delta_log"
```

```
berkas_log = sorted(f for f in os.listdir(PATH_LOG) if f.endswith(".json"))
print("berkas log:", berkas_log)

for nama in berkas_log:
    print("\n" + "=" * 70)
    print(nama)
    print("=" * 70)
    with open(f"{PATH_LOG}/{nama}", encoding="utf-8") as f:
        for baris in f:
            print(baris.rstrip())
```

Output tersimpan

Isi lengkap empat berkas JSON tercetak pada notebook tugas yang dapat diakses melalui tautan repository dan Google Drive di lampiran. Ringkasan aksi log ditampilkan pada cell berikutnya.

Cell T20. Ringkasan Aksi Log (cell notebook 37)

```
ringkas = []
for nama in berkas_log:
    ditambah, dihapus, operasi = 0, 0, None
    with open(f"{PATH_LOG}/{nama}", encoding="utf-8") as f:
        for baris in f:
            aksi = json.loads(baris)
            if "add" in aksi:
                ditambah += 1
            elif "remove" in aksi:
                dihapus += 1
            elif "commitInfo" in aksi:
                operasi = aksi["commitInfo"].get("operation")
    ringkas.append({"berkas": nama, "operasi": operasi, "file ditambah": ditambah, "file dihapus": dihapus})

display(pd.DataFrame(ringkas))
```

Output tersimpan

	berkas operasi	file ditambah	file dihapus	
0	00000000000000000000.json	WRITE	1	0
1	00000000000000000001.json	WRITE	1	0
2	00000000000000000002.json	WRITE	1	0
3	00000000000000000003.json	UPDATE	1	3

Cell T21. Membuat Arsip Delta Dua Hari (cell notebook 39)

```
PATH_ZIP_2HARI = f"{DIR_SIMPAN}/trips_delta_2hari"
hasil_zip = shutil.make_archive(PATH_ZIP_2HARI, "zip", root_dir=PATH_2HARI)
print("Arsip tersimpan:", hasil_zip)
```

Output tersimpan

Arsip tersimpan: /content/drive/MyDrive/BigData/Praktikum4/TugasMandiri/trips_delta_2hari.zip

Sampel 100.000 baris dibagi menjadi batch pagi 40.000 dan sore 20.000 pada hari pertama. Akhir hari pertama berada pada versi 1 dengan 60.000 baris dan total pembayaran 1.559.049,94. Hari kedua menambah 40.000 baris lalu mengoreksi 704 perjalanan Tunai di Queens sebesar 1,0, menghasilkan versi 3 dengan 100.000 baris.

Pembacaan ulang versi 1 mengembalikan jumlah dan total pembayaran akhir hari pertama dengan pemeriksaan True. Selisih total terbaru terhadap hari pertama adalah 1.038.019,97, sama dengan total batch hari kedua ditambah koreksi. Pemeriksaan tersebut memvalidasi jumlah dan total, bukan perbandingan setiap elemen seluruh baris. Empat log JSON mencatat tiga WRITE dan satu UPDATE. UPDATE menambahkan satu file dan mengeluarkan tiga file dari keadaan aktif tabel.

5.5 Ketentuan Pengumpulan R-4

Modul menetapkan pengumpulan mandiri melalui iLearn paling lambat tujuh hari setelah praktikum. Berkas wajib adalah notebook .ipynb dengan output, benchmark_nosql.csv, trips_delta.zip, dan laporan PDF. Studi kasus diminta sepanjang 2 sampai 3 halaman; Bagian 4.11 disusun dalam tiga halaman. Modul juga meminta Restart notebook and run all pada runtime bersih.

Berkas atau pemeriksaan	Status bukti pada lampiran
Notebook utama dan tugas	Kedua notebook tersedia; banyak output tersimpan.
benchmark_nosql.csv	Kode penyimpanan dan isi 19 baris tercetak; file CSV terpisah tidak dilampirkan.
rekap_k9.csv	Output mencatat path penyimpanan pada TugasMandiri.
trips_delta.zip	Output mencatat path arsip; file ZIP terpisah tidak dilampirkan.
trips_delta_2hari.zip	Output mencatat arsip tugas mandiri.
Restart dan Run all	Tidak ada bukti eksplisit sesi bersih; cell verifikasi Q-2 belum memiliki output.

5.6 Pertanyaan Evaluasi

1. Mengapa Not Only SQL dan kapan relasional lebih tepat

NoSQL melengkapi pilihan penyimpanan sesuai bentuk data dan query. Relasional tetap tepat untuk transaksi pembayaran yang membutuhkan konsistensi dan keterkaitan data, serta laporan terstruktur yang sering memakai JOIN dan GROUP BY. K-2 memperlihatkan bahwa tabel berindeks juga cepat untuk mencari ID.

2. Perbedaan BASE ACID dan CAP

BASE adalah Basically Available, Soft state, Eventually consistent; hasil pada salinan dapat memerlukan waktu untuk menjadi konsisten. ACID menekankan transaksi atomik, keadaan yang memenuhi aturan konsistensi, isolasi, dan ketahanan data setelah komit. Saat partisi jaringan terjadi, CAP membahas pilihan antara konsistensi dan ketersediaan. Saldo pembayaran perlu mengutamakan konsistensi; daftar perjalanan terbaru dapat menerima keterlambatan yang disepakati. Pilihan ini tidak diuji pada runtime tunggal praktikum.

3. Pencarian cepat dan agregasi lambat pada key-value

Dict mengakses nilai melalui kunci sehingga pencarian tidak memeriksa setiap baris. Namun nilai JSON tidak menyediakan struktur kolom untuk agregasi; setiap nilai perlu diurai. Kesederhanaan pola akses mengorbankan kemudahan query berdasarkan isi, hubungan, dan agregasi tanpa struktur tambahan.

4. Embedding dan referencing

Embedding menyimpan data terkait pada satu dokumen, sesuai profil yang dibaca bersama dan jarang berubah. Referencing menyimpan ID yang menunjuk data lain, sesuai informasi yang digunakan banyak objek dan sering diperbarui. Embedding dapat menduplikasi informasi wilayah, sedangkan referencing membutuhkan pencarian tambahan.

5. Kapan graph lebih layak dari SQL

Graph layak untuk eksplorasi tetangga, jalur, dan jumlah langkah yang berubah. API keterjangkauan langsung mewakili struktur jaringan. K-5 menunjukkan 74 zona pada satu langkah dan 214 pada paling banyak dua hingga empat langkah. Kemudahan representasi dan pengembangan query menjadi alasan selain waktu; SQL rekursif juga dapat menjawab keterhubungan.

6. Tahap awal indeks spasial dan klaim percepatan

Indeks lebih dahulu menyaring kandidat melalui kotak pembatas, lalu mengevaluasi hubungan geometris tepat. Klaim ribuan kali dapat menyedatkan jika membandingkan loop Python dengan operasi indeks yang telah dibangun, mengabaikan NumPy, atau memakai jumlah kotak berbeda. Pada K-6, keuntungan terhadap NumPy hanya 1,78 kali termasuk pembangunan indeks; pada kotak besar di Latihan 5 STRtree bahkan lebih lambat.

7. Isi log Delta atomisitas time travel dan VACUUM

Folder `_delta_log/` memuat catatan komit seperti `commitInfo`, `metadata`, `protocol`, `add`, dan `remove`. Pembaca menentukan file aktif berdasarkan versi log yang telah dikomit; perubahan yang belum menjadi komit lengkap tidak menjadi keadaan tabel yang baru. Versi lama dapat dibaca selama log dan file yang dibutuhkan masih ada. VACUUM nyata dengan retention nol dapat menghapus file versi lama atau file yang masih dibaca proses lain; K-8 hanya menggunakan dry run.

BAB VI

PENUTUP

6.1 Kesimpulan

Pola query menentukan kesesuaian penyimpanan. Pada sampel 300.000 baris, indeks menurunkan waktu pencarian 200 ID dari 4,3215 menjadi 0,0037 detik, mendekati key-value 0,0012 detik. Untuk agregasi, SQL lebih cepat daripada pembacaan seluruh nilai JSON. Pada seluruh data, DuckDB menyelesaikan query per wilayah dalam 0,1139 detik, dibanding SQLite 1,7269 detik dengan hasil sama.

Document mendukung field opsional dan penambahan atribut, sedangkan graph menyederhanakan eksplorasi keterhubungan. STRtree mempercepat pencarian kotak kecil pada titik sintetis, tetapi besar keuntungannya bergantung pada pembandingan dan ukuran kotak. Delta Lake mempertahankan versi, menolak skema yang berbeda, serta mengurangi file aktif melalui compaction tanpa mengubah jumlah baris. Restore membentuk versi baru sehingga riwayat audit tetap dapat ditelusuri.

DAFTAR PUSTAKA

- Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., Świtakowski, M., Szafranski, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., ... Zaharia, M. (2020). Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12), 3411-3424.
<https://doi.org/10.14778/3415478.3415560>
- Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. *Computer*, 45(2), 23-29.
- delta-rs. (t.t.). *The deltalake Python package documentation*. <https://delta-io.github.io/delta-rs/>
- DuckDB. (t.t.). *Documentation*. <https://duckdb.org/docs/>
- Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. Dalam *Proceedings of ACM SIGMOD*. Association for Computing Machinery.
- Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.
- Modul Praktikum Big Data: Praktikum 4, NoSQL dan Pemodelan Data*. (t.t.).
- NYC Taxi & Limousine Commission. (t.t.). *TLC trip record data*.
<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- Pritchett, D. (2008). BASE: An ACID alternative. *ACM Queue*, 6(3), 48-55.
- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. Dalam *Proceedings of the 2019 International Conference on Management of Data* (hlm. 1981-1984). Association for Computing Machinery.
<https://doi.org/10.1145/3299869.3320212>

Rizqullah, H. P. (2026a). *Praktikum Big Data 4: NoSQL dan pemodelan data* [Notebook Google Colab].

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum04

Rizqullah, H. P. (2026b). *Tugas Praktikum Big Data 4: NoSQL dan pemodelan data* [Notebook Google Colab].

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum04

Sadalage, P. J., & Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

LAMPIRAN

Repository Praktikum04: https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum04

Google Drive:

https://drive.google.com/drive/folders/1KURZQOGDvuQaNNX6q9Af6vrRw3Zcnfmt?usp=drive_link